

Geometric and 3D Vision

Depth Estimation, Stereo, Camera Pose, and SLAM

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية

King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

October 7, 2026

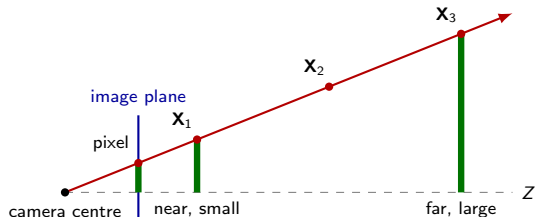
1. Motivation: what a camera discards, four problems, and where geometry meets learning
2. Camera geometry: homogeneous coordinates, the pinhole model, intrinsics and extrinsics, distortion, homographies, and calibration
3. Two-view geometry: local features, the epipolar constraint, essential and fundamental matrices, RANSAC, and triangulation
4. Stereo: rectification, disparity and depth error, cost volumes, semi-global matching, learned stereo, and active sensors
5. Monocular depth: scale ambiguity, supervised and self-supervised training, relative and metric depth, and diffusion priors
6. Camera pose and structure from motion: PnP, visual localization, pose regression, bundle adjustment, and global SfM
7. SLAM: drift and loop closure, feature-based, direct, and learned SLAM, and dense maps
8. Feed-forward 3D reconstruction: pointmaps, DUST3R, MAST3R, VGGT, and their successors
9. Limitations, summary, and references

- ▶ **Covered earlier:** Depth Anything predicts a depth map from a single image, and SuperGlue matches keypoints between two images. Both were used as pretrained components. This lecture supplies the geometry that turns such outputs into cameras, depths, and maps.
- ▶ **Also covered earlier:** optical flow as a dense motion field, RGB-D cameras and LiDAR as depth sensors, and image retrieval with global descriptors. Each returns here as one stage of a 3D pipeline.
- ▶ **Backbones we reuse:** ViT and cross-attention, DINOv2 features, masked autoencoders, and Stable Diffusion. The learned depth and 3D models later in the lecture are assembled from these parts.
- ▶ **Route:** camera model $\rightarrow$ two views $\rightarrow$ stereo $\rightarrow$ single-image depth $\rightarrow$ pose and structure from motion $\rightarrow$ SLAM $\rightarrow$ feed-forward 3D models.

By the end of this lecture you should be able to:

- ▶ **Write** the pinhole projection and back-projection, and explain calibration and homographies.
- ▶ **Derive** the epipolar constraint and recover relative pose and 3D points despite outliers.
- ▶ **Compute** depth from disparity and its error, and explain cost volumes and learned stereo.
- ▶ **Compare** monocular depth estimators by their training signal and evaluation protocol.
- ▶ **Estimate** absolute camera pose and explain structure from motion with bundle adjustment.
- ▶ **Describe** a visual SLAM system and contrast feature-based, direct, and learned SLAM.
- ▶ **Explain** how feed-forward 3D models replace parts of the classical pipeline, and their limits.
- ▶ **Assumed background** (all covered earlier): CNN encoder-decoders, GRUs, 3D convolutions for video, ViT and attention, DINOv2, masked autoencoders, latent diffusion and DDIM sampling, optical flow, image retrieval with nearest-neighbour search, SVD, Depth Anything, and SuperGlue.

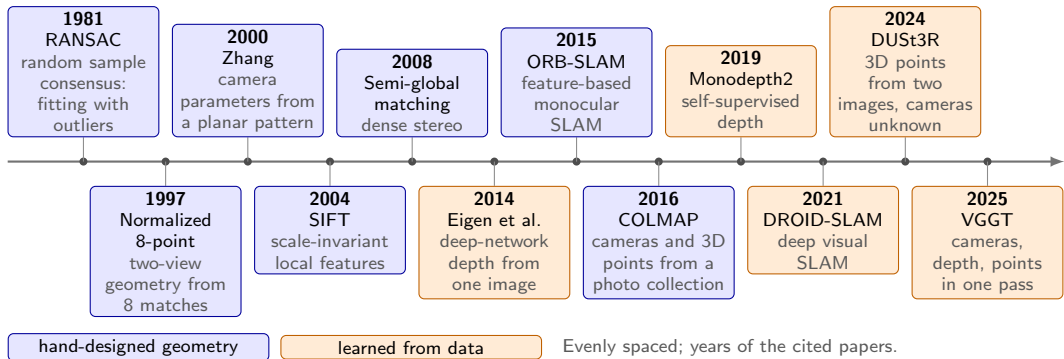
- ▶ A camera maps the 3D world onto a 2D image and discards depth: every point on a ray through the camera centre lands on the same pixel.
- ▶ Size and distance trade off. An object twice as tall and twice as far away covers the same pixels, so one image cannot separate a small near object from a large far one.
- ▶ Many tasks need metric 3D, in metres: robot navigation and grasping, autonomous driving (the distance to the car ahead), augmented reality (AR) content that stays anchored to real surfaces, and mapping or inspection of buildings and sites.
- ▶ Getting depth back needs extra information: a second view, camera motion, an object of known size, or priors learned from data.



X_1 , X_2 and X_3 lie on one ray and image to the same pixel. The far object (green) is three times taller and three times farther away than the near one, so their images coincide.

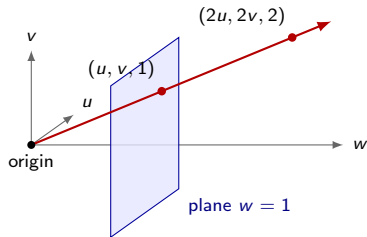
Depth estimation	Stereo	Camera pose	SLAM
In: one image (or a video) Out: a depth for every pixel Unknown: the distance along each pixel's ray	In: two cameras a fixed distance apart, with known lens parameters and relative pose Out: dense depth Unknown: which pixel in one view matches each pixel in the other	In: images, with or without a 3D map Out: position and orientation of each camera Unknown: where each image was taken (and the 3D points, if no map exists)	In: a live video stream Out: the camera trajectory and a map, updated online Unknown: pose and map at the same time, with no map given in advance
Correspondences: the same 3D point identified in several images, or a learned prior when only one image exists			
Projection model: how a 3D point becomes a pixel (camera geometry, next section)			

- ▶ SLAM stands for simultaneous localization and mapping. All four problems invert the same projection and differ in what is known: the cameras, the scene, or neither.
- ▶ Learned priors increasingly replace hand-designed steps: learned features and matchers, networks that predict depth from a single image, and models that output cameras and 3D points in one forward pass.



- The lecture follows this arc: the geometry first, then the learned components that replace parts of it.

- ▶ **Homogeneous coordinates:** write the pixel (u, v) as the 3-vector $(wu, wv, w)^T$ for any $w \neq 0$. All of them are the same point, $\tilde{\mathbf{x}} = (u, v, 1)^T \sim (wu, wv, w)^T$, where $\sim$ means equal up to a nonzero scale.
- ▶ Back to pixels: divide by the last entry, $(x_1, x_2, x_3)^T \mapsto (x_1/x_3, x_2/x_3)$. Example: $(6, 4, 2)^T \sim (3, 2, 1)^T$ is the pixel $(3, 2)$.
- ▶ **Points at infinity** have $w = 0$: $(1, 2, 0)^T$ is the direction $(1, 2)$, the point where all lines with that direction meet.
- ▶ **Lines** are 3-vectors too: $\mathbf{l} = (a, b, c)^T$ is the line $au + bv + c = 0$, so $\tilde{\mathbf{x}}$ lies on it exactly when $\mathbf{l}^T \tilde{\mathbf{x}} = 0$. Two lines meet at $\mathbf{l}_1 \times \mathbf{l}_2$: the parallel lines $u = 1$ and $u = 2$, i.e. $(1, 0, -1)^T$ and $(1, 0, -2)^T$, meet at $(0, 1, 0)^T$, the point at infinity in the v direction.
- ▶ **Why:** perspective division becomes a matrix product (next slide), and a rotation plus a translation of a 3D point, written as the 4-vector $(X, Y, Z, 1)^T$, becomes a single 4×4 matrix.



Every homogeneous vector is a ray through the origin; the pixel is where that ray meets the plane $w = 1$. After Hartley and Zisserman (2004), Ch. 2.

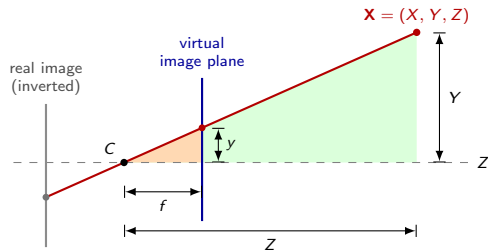
- **Pinhole model:** every ray from a scene point $\mathbf{X} = (X, Y, Z)^\top$ passes through the optical centre C and meets the image plane at distance f (the focal length) behind C , upside down. A virtual image plane at f in front of C gives the same image unflipped; later figures use it.
- Camera coordinates: origin at C , Z along the optical axis (the line through C perpendicular to the image plane), X and Y parallel to the image axes. Similar triangles give

$$x = f \frac{X}{Z}, \quad y = f \frac{Y}{Z} \quad (Z > 0),$$

with (x, y) in the units of f , e.g. millimetres.

- In homogeneous coordinates the division becomes the scale, with $\tilde{\mathbf{X}} = (X, Y, Z, 1)^\top$:

$$\begin{pmatrix} x \\ y \\ 1 \end{pmatrix} \sim \begin{pmatrix} fX \\ fY \\ Z \end{pmatrix} = \begin{bmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \tilde{\mathbf{X}}.$$



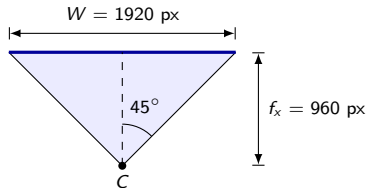
Side view. The ray from $\mathbf{X}$ through C meets the virtual plane at $y = fY/Z$ and the real image at $-fY/Z$; the two shaded triangles are similar. The depth Z is divided out, so a single pixel cannot return it. Camera model after Hartley and Zisserman (2004), Ch. 6.

- Pixels are not millimetres: each sensor pixel has a pitch (size) $\rho_x \times \rho_y$, and pixel (0,0) sits at the top-left corner, away from the optical axis.

- **Intrinsics K :** with the depth $\lambda = Z$,

$$\lambda \begin{pmatrix} u \\ v \\ 1 \end{pmatrix} = \underbrace{\begin{bmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}}_K \begin{pmatrix} X \\ Y \\ Z \end{pmatrix}.$$

- $f_x = f/\rho_x$ and $f_y = f/\rho_y$: focal lengths in pixels. (c_x, c_y) : the principal point, where the optical axis meets the image, usually near its centre. s : skew between the pixel axes, ≈ 0 for most cameras.
- K has 5 parameters and depends only on the camera itself (lens, sensor, zoom and focus setting); moving the camera leaves it unchanged.
- **Field of view:** with the principal point at the centre of a W -pixel-wide image, the horizontal field of view is $\text{HFOV} = 2 \arctan(W/(2f_x))$.



Worked example (top view): a 1920-pixel-wide image with $\text{HFOV} = 90^\circ$ has

$$f_x = \frac{W/2}{\tan(\text{HFOV}/2)} = \frac{960}{\tan 45^\circ} = 960 \text{ px}.$$

The same camera from its hardware: a 3.0 mm lens on pixels of pitch $3.125 \mu\text{m}$ gives $f_x = 3.0 \text{ mm}/3.125 \mu\text{m} = 960 \text{ px}$, and the 1920 pixels span 6.0 mm.

- ▶ **Extrinsics** $[R \mid \mathbf{t}]$: a rigid motion takes world coordinates to camera coordinates,

$$\mathbf{X}_c = R \mathbf{X}_w + \mathbf{t}.$$

$R \in SO(3)$, the group of 3D rotations ($R^\top R = I$, $\det R = 1$), has 3 degrees of freedom (DoF); $\mathbf{t} \in \mathbb{R}^3$ has 3 more.

- ▶ In homogeneous form, $\tilde{\mathbf{X}}_c = T \tilde{\mathbf{X}}_w$ with $T = \begin{bmatrix} R & \mathbf{t} \\ \mathbf{0}^\top & 1 \end{bmatrix} \in SE(3)$, the group of rigid motions.
- ▶ **Camera centre:** $\mathbf{X}_c = \mathbf{0}$ gives $\mathbf{C} = -R^\top \mathbf{t}$ in world coordinates; $\mathbf{t}$ is the world origin expressed in camera coordinates.
- ▶ **Projection:**

$$\lambda \tilde{\mathbf{x}} = K [R \mid \mathbf{t}] \tilde{\mathbf{X}} = P \tilde{\mathbf{X}}.$$

P is 3×4 and defined up to scale: $12 - 1 = 11$ DoF, 5 intrinsic plus 6 extrinsic. $\lambda = Z_c$ is the depth; $\pi(K, R, \mathbf{t}, \mathbf{X})$ denotes the resulting pixel $\mathbf{x}$.

Worked example

$f_x = f_y = 500$, $s = 0$, $(c_x, c_y) = (320, 240)$; $R = I$, $\mathbf{t} = (0, 0, 4)^\top$; world point $\mathbf{X}_w = (1, 0.5, 0)^\top$, in metres.

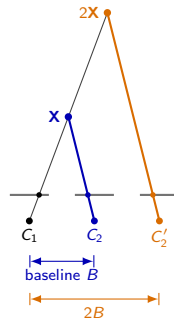
1. $\mathbf{X}_c = R \mathbf{X}_w + \mathbf{t} = (1, 0.5, 4)^\top$
2. $K \mathbf{X}_c = (500 \cdot 1 + 320 \cdot 4, 500 \cdot 0.5 + 240 \cdot 4, 4)^\top = (1780, 1210, 4)^\top$
3. Divide by $\lambda = 4$: $(u, v) = (445, 302.5)$
4. $\mathbf{C} = -R^\top \mathbf{t} = (0, 0, -4)^\top$: the camera is 4 m behind the world origin.

- **Back-projection** inverts the projection for a pixel whose depth Z is known ($s = 0$):

$$\mathbf{X}_c = Z K^{-1} \tilde{\mathbf{x}} = \left(Z \frac{u - c_x}{f_x}, Z \frac{v - c_y}{f_y}, Z \right)^T,$$

and $\mathbf{X}_w = R^T (\mathbf{X}_c - \mathbf{t})$ in world coordinates. Check: (445, 302.5) at $Z = 4$ gives $(4 \cdot 125/500, 4 \cdot 62.5/500, 4) = (1, 0.5, 4)$.

- Without Z a pixel fixes only a ray, $\{\mu K^{-1} \tilde{\mathbf{x}} : \mu > 0\}$. Depth means the Z coordinate, measured along the optical axis; the distance along an off-axis ray is larger.
- **Scale ambiguity:** scaling the scene and the translation by the same $\alpha > 0$ changes no pixel, since $K[R | \alpha \mathbf{t}] (\alpha \mathbf{X}_w, 1)^T = \alpha K[R | \mathbf{t}] \tilde{\mathbf{X}}$. One moving camera therefore recovers depth only up to scale. A stereo pair with a known distance between the cameras (the baseline), an object of known size, or an IMU (inertial measurement unit: accelerometers and gyroscopes) fixes it.
- A depth map $D(\mathbf{p})$, one depth per pixel $\mathbf{p}$, back-projects to a **point cloud**: an unordered set of 3D points $\{D(\mathbf{p}) K^{-1} \tilde{\mathbf{p}}\}$, often with each pixel's colour.

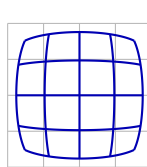


Top view, short bars are image planes. Doubling the scene and the baseline (orange) gives the same pixels: the orange ray is parallel to the blue one, and the black ray is shared.

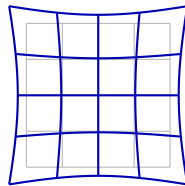
- ▶ **Lens distortion:** a real lens departs from the pinhole's central projection, mostly because its magnification changes with the distance from the principal point, so straight lines can image as curves. The pinhole model holds only after this is removed.
- ▶ On normalized coordinates $(x, y) = (X/Z, Y/Z)$, the pinhole image for $f = 1$, with $r^2 = x^2 + y^2$:

$$\begin{aligned}x_d &= x(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + 2p_1 xy + p_2(r^2 + 2x^2), \\y_d &= y(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + p_1(r^2 + 2y^2) + 2p_2 xy, \\u &= f_x x_d + c_x, \quad v = f_y y_d + c_y \quad (s = 0 \text{ as in OpenCV}).\end{aligned}$$

- ▶ **Radial** terms k_1, k_2, k_3 move points along the radius, more towards the border: barrel distortion when $1 + k_1 r^2 + k_2 r^4 + k_3 r^6$ decreases with r (usually $k_1 < 0$), pincushion when it increases.
- ▶ **Tangential** (decentering) terms p_1, p_2 come from lens elements that are not exactly centred and parallel to the sensor.
- ▶ **Calibration** estimates K and these coefficients (next slides); images or keypoints are then undistorted by inverting this map numerically.



barrel, $k_1 = -0.12$



pincushion, $k_1 = +0.12$

Grey: a square grid of half-width 1 in normalized coordinates. Blue: its image under the radial term k_1 alone. A real camera with a 6 mm lens, calibrated by Zhang (MSR-TR-98-71, Table 1), has $k_1 = -0.228$, $k_2 = 0.190$: barrel.

Model: Brown (1971), Eq. 20, written there as the inverse correction; forward form and names from OpenCV calib3d, docs.opencv.org.

- ▶ A **homography** is an invertible 3×3 matrix on homogeneous pixels, $\tilde{x}' \sim H\tilde{x}$, defined up to scale (8 DoF). It maps lines to lines.
- ▶ **Plane-induced homography**: put the world plane at $Z = 0$. The third column r_3 of $R = [r_1 \ r_2 \ r_3]$ then drops out:

$$\lambda \tilde{x} = K[r_1 \ r_2 \ r_3 \ t] (X, Y, 0, 1)^T = \underbrace{K[r_1 \ r_2 \ t]}_H (X, Y, 1)^T.$$

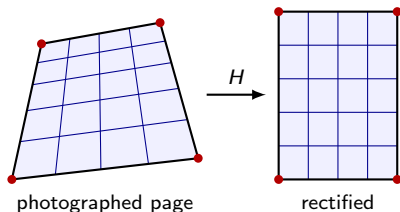
Two views of the same plane are therefore related by $H_2 H_1^{-1}$.

- ▶ **Pure rotation** (no translation, as for a panorama): $\tilde{x}' \sim KRK^{-1}\tilde{x}$ for every point, whatever its depth.
- ▶ **DLT** (direct linear transform): with $\mathbf{a}^T, \mathbf{b}^T, \mathbf{c}^T$ the rows of H , $u' = \mathbf{a}^T \tilde{x} / \mathbf{c}^T \tilde{x}$ and $v' = \mathbf{b}^T \tilde{x} / \mathbf{c}^T \tilde{x}$ give two equations per match,

$$\mathbf{a}^T \tilde{x} - u' \mathbf{c}^T \tilde{x} = 0, \quad \mathbf{b}^T \tilde{x} - v' \mathbf{c}^T \tilde{x} = 0,$$

linear in the 9 entries $\mathbf{h}$ of H . Stack $n \geq 4$ matches into $A\mathbf{h} = \mathbf{0}$ (A is $2n \times 9$); $\mathbf{h}$ is the right singular vector of A with the smallest singular value (SVD, covered earlier). First translate and scale each image's points to zero mean and mean distance $\sqrt{2}$ (matrices T, T'), solve for $\hat{H}$, then undo it: $H = T'^{-1} \hat{H} T$.

- ▶ Wrong matches (outliers) ruin this fit; RANSAC (random sample consensus, two-view section) removes them.



Four corner matches (red) fix H , and every grid line of the photographed page lands on its place in the upright page. A bird's-eye view of a road applies the same mapping to the road plane. After Hartley and Zisserman (2004), Ch. 4 and 13.

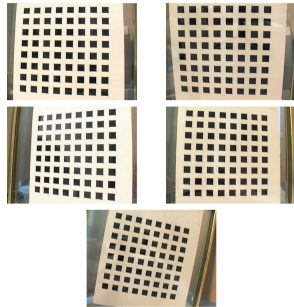
- ▶ Zhang's calibration method uses a flat pattern of known geometry, such as a printed checkerboard, as the plane $Z = 0$, so every corner has known coordinates $(X, Y, 0)$.
- ▶ The camera sees the pattern "at a few (at least two) different orientations"; the camera or the board may move, and the motion need not be known.
- ▶ **Per view i :** detect the corners and fit the plane-induced $H_i = [\mathbf{h}_1 \ \mathbf{h}_2 \ \mathbf{h}_3] \sim K[\mathbf{r}_1 \ \mathbf{r}_2 \ \mathbf{t}]$ by DLT ($\mathbf{r}_1, \mathbf{r}_2, \mathbf{t}$ of view i , index dropped). Since $\mathbf{r}_1 \perp \mathbf{r}_2$ and $\|\mathbf{r}_1\| = \|\mathbf{r}_2\|$,

$$\mathbf{h}_1^\top \omega \mathbf{h}_2 = 0, \quad \mathbf{h}_1^\top \omega \mathbf{h}_1 = \mathbf{h}_2^\top \omega \mathbf{h}_2, \quad \omega = K^{-\top} K^{-1}.$$

- ▶ **Closed form:** two linear equations per view in the 6 entries of the symmetric ω , solved by SVD; 3 views suffice in general, 2 with zero skew. Then K follows from ω , and $R_i, \mathbf{t}_i$ from each H_i .
- ▶ **Refinement:** nonlinear least squares over K , the distortion coefficients $\mathbf{k}$ and all poses, minimizing the squared **reprojection error**, the pixel distance between each observed corner $\mathbf{x}_{ij}$ and the projection of its known point $\mathbf{X}_j$, with the distortion map (coefficients $\mathbf{k}$) applied before K :

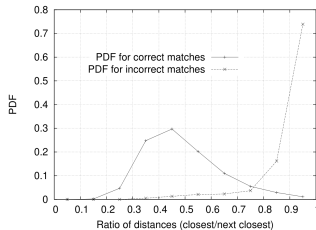
$$\sum_i \sum_j \|\mathbf{x}_{ij} - \pi(K, \mathbf{k}, R_i, \mathbf{t}_i, \mathbf{X}_j)\|^2.$$

On Zhang's 640×480 camera with 5 views, the root-mean-square (RMS) error fell from 0.881 px (closed form) to 0.335 px.



The five views of Zhang's real-data experiment:
8 × 8 separate squares, 256 corners, 17 cm wide. Z. Zhang, "A Flexible New Technique for Camera Calibration," IEEE TPAMI 2000, doi:10.1109/34.888718; figure from the report MSR-TR-98-71, Figure 4.

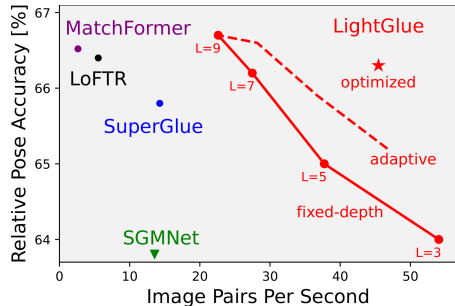
- ▶ **Detect** repeatable points: the same corner or blob is found again from another viewpoint.
- ▶ **Describe** each point by a vector computed from its neighbourhood, built to change little under rotation, scale and illumination. **Match** by nearest neighbour in descriptor space.
- ▶ **SIFT** (Scale Invariant Feature Transform; Lowe, 2004): keypoints are extrema of a difference-of-Gaussians scale space (differences of successively blurred copies of the image). The descriptor holds gradient-orientation histograms over 4×4 cells $\times$ 8 bins = 128 values, measured relative to the keypoint's dominant orientation.
- ▶ **ORB** (Oriented FAST and Rotated BRIEF; Rublee et al., 2011): FAST corners (an intensity test on a ring of pixels) given an orientation, and a 256-bit string of pixel-pair intensity comparisons rotated with the keypoint, compared by Hamming distance (the number of differing bits). On the same 640×480 images with about 1000 features: 15.3 ms per frame against 5228.7 ms for SIFT.



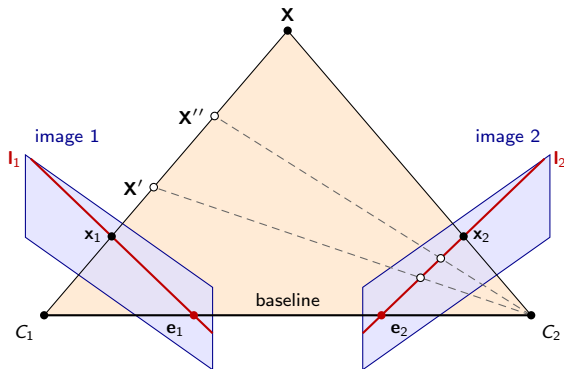
Lowe, "Distinctive Image Features from Scale-Invariant Keypoints," IJCV 60(2), 2004, doi:10.1023/B:VISI.0000029664.99615.94, Figure 11.

Ratio test: keep the nearest neighbour only if $d_1/d_2 < 0.8$, where d_1 and d_2 are the distances to the closest and second-closest descriptors. Correct matches (solid curve) have low ratios; Lowe reports that the 0.8 threshold "eliminates 90% of the false matches while discarding less than 5% of the correct matches."

- ▶ **SuperPoint** (DeTone et al., 2018): one fully convolutional network, a shared encoder with two heads (keypoint heatmap, 256-D descriptors), run once per image. Self-supervised: a detector trained on rendered synthetic shapes with known corners labels real images by *homographic adaptation*, pooling its detections over many random homographic warps of each image.
- ▶ **SuperGlue** (covered earlier) matches two keypoint sets jointly with self- and cross-attention and a Sinkhorn assignment.
- ▶ **LightGlue** (Lindenberg et al., 2023) adapts its computation to the pair. After each layer it predicts how confident each point is, stops once enough points are confident (depth), and drops points it is sure are unmatchable (width). For pose estimation on MegaDepth-1500 (1500 outdoor image pairs): 31.4 ms per pair against 70.0 ms for SuperGlue, at similar accuracy.
- ▶ **LoFTR** (Local Feature TRansformer; Sun et al., 2021) has no detector. Self- and cross-attention over a dense grid of 1/8-resolution features give coarse matches (mutual nearest neighbours), each refined to sub-pixel in a window of 1/2-resolution features. It finds matches on texture-less walls, where detectors struggle to find repeatable points.
- ▶ **RoMa** (Edstedt et al., 2024): dense matching, a warp plus a certainty for every pixel, from frozen DINOv2 coarse features (covered earlier) and ConvNet fine features.



“LightGlue matches sparse features faster and better than existing approaches like SuperGlue.”
Lindenberg et al., “LightGlue: Local Feature Matching at Light Speed,” ICCV 2023, [arXiv:2306.13643v1](https://arxiv.org/abs/2306.13643v1), Figure 1.



- ▶ The baseline (defined above) joins the camera centres C_1 and C_2 . Together with a scene point X it spans the **epipolar plane** (shaded).
- ▶ The **epipoles** e_1, e_2 are where the baseline pierces the image planes: e_2 is the image of C_1 in view 2, and e_1 the image of C_2 in view 1.
- ▶ The epipolar plane cuts the image planes in the **epipolar lines** l_1, l_2 . Every point on the ray through x_1 (X, X', X'') lies in that plane, so its image in view 2 lies on l_2 .
- ▶ The search for the match of x_1 shrinks from the whole image (2D) to the line l_2 (1D). All epipolar lines of an image pass through its epipole.

- ▶ Take camera 1 as the world frame: a point at $\mathbf{X}_1$ in camera 1 is at $\mathbf{X}_2 = R\mathbf{X}_1 + \mathbf{t}$ in camera 2, so $\mathbf{t}$ is the position of C_1 seen from C_2 .
- ▶ The normalized coordinates (defined above), written homogeneously as $\hat{\mathbf{x}}_i = K_i^{-1}\tilde{\mathbf{x}}_i = (x_i, y_i, 1)^\top$, are ray directions: $\mathbf{X}_i = Z_i \hat{\mathbf{x}}_i$, with Z_i the depth in camera i .
- ▶ In camera-2 axes, $\hat{\mathbf{x}}_2$, $\mathbf{t}$ and $R\hat{\mathbf{x}}_1$ all lie in the epipolar plane, so they are coplanar: $\hat{\mathbf{x}}_2 \cdot (\mathbf{t} \times R\hat{\mathbf{x}}_1) = 0$, that is

$$\hat{\mathbf{x}}_2^\top E \hat{\mathbf{x}}_1 = 0, \quad E = [\mathbf{t}]_\times R,$$

with the cross-product matrix $[\mathbf{t}]_\times = \begin{bmatrix} 0 & -t_3 & t_2 \\ t_3 & 0 & -t_1 \\ -t_2 & t_1 & 0 \end{bmatrix}$,

so that $[\mathbf{t}]_\times \mathbf{a} = \mathbf{t} \times \mathbf{a}$ for any vector $\mathbf{a}$.

- ▶ Substituting $\hat{\mathbf{x}}_i = K_i^{-1}\tilde{\mathbf{x}}_i$ moves the constraint to pixels: $\tilde{\mathbf{x}}_2^\top F \tilde{\mathbf{x}}_1 = 0$ with $F = K_2^{-\top} E K_1^{-1}$.

	Essential E	Fundamental F
Points	normalized $\hat{\mathbf{x}}$ (K known)	pixels $\tilde{\mathbf{x}}$
Degrees of freedom	5: 3 for R , 3 for $\mathbf{t}$, minus 1 for scale	7: 9 entries, minus 1 for scale, minus 1 for $\det F = 0$
Rank	2, singular values $(\sigma, \sigma, 0)$	2

- ▶ **Epipolar line** of $\tilde{\mathbf{x}}_1$ in image 2: $\mathbf{l}_2 = F\tilde{\mathbf{x}}_1$, so its match satisfies $\tilde{\mathbf{x}}_2^\top \mathbf{l}_2 = 0$.
- ▶ **Epipoles** are the null vectors: $F\mathbf{e}_1 = \mathbf{0}$ and $F^\top \mathbf{e}_2 = \mathbf{0}$.

Hartley and Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed., 2004, [doi:10.1017/CBO9780511811685](https://doi.org/10.1017/CBO9780511811685), Table 9.1 and Section 9.6.

- ▶ A match $(u_1, v_1) \leftrightarrow (u_2, v_2)$ turns $\tilde{\mathbf{x}}_2^\top F \tilde{\mathbf{x}}_1 = 0$ into one linear equation in the 9 entries $\mathbf{f} = (F_{11}, F_{12}, \dots, F_{33})^\top$:

$$(u_2 u_1, u_2 v_1, u_2, v_2 u_1, v_2 v_1, v_2, u_1, v_1, 1) \mathbf{f} = 0.$$

- ▶ Stack $n \geq 8$ matches into $A\mathbf{f} = \mathbf{0}$, with A of size $n \times 9$. The minimizer of $\|A\mathbf{f}\|$ subject to $\|\mathbf{f}\| = 1$ is the right singular vector of A for its smallest singular value.
- ▶ **Enforce rank 2:** from $F = U \text{diag}(\sigma_1, \sigma_2, \sigma_3) V^\top$ keep $U \text{diag}(\sigma_1, \sigma_2, 0) V^\top$, the closest singular matrix in Frobenius norm (root of the sum of squared entries). Without it the epipolar lines do not meet in a single epipole.

Hartley, "In Defense of the Eight-Point Algorithm," TPAMI 19(6), 1997, doi:10.1109/34.601246. Nistér, "An Efficient Solution to the Five-Point Relative Pose Problem," TPAMI 26(6), 2004, doi:10.1109/TPAMI.2004.17.

- ▶ **Normalize first** (Hartley, 1997), with translate-and-scale matrices T_i as for the homography DLT: estimate $\hat{F}$ from the points $T_i \tilde{\mathbf{x}}_i$, then undo it: $F = T_2^\top \hat{F} T_1$.
- ▶ **Why:** with pixel coordinates near 100, a row of A mixes entries near 10^4 with entries equal to 1, so $A^\top A$ has entries from 10^8 down to 1 and a condition number (eigenvalue ratio) of at least about 10^4 . Small noise then swings the solution; after normalization the linear algorithm is comparable with the best iterative ones.
- ▶ **Minimal solvers:** 7 matches leave a 2-D null space, $F = \alpha F_1 + (1 - \alpha) F_2$, and $\det F = 0$ is a cubic in α : up to 3 solutions. With K known, 5 matches fix E (Nistér, 2004) as roots of a tenth-degree polynomial: up to 10 solutions.

RANSAC (random sample consensus; Fischler and Bolles, 1981) repeats N times:

1. Sample a minimal set of s matches at random.
 2. Fit the model to them: four-point homography, five-point E , seven- or eight-point F .
 3. Count the **inliers**: matches whose error is below a threshold τ (for F , the distance from $\tilde{x}_2$ to its epipolar line $F\tilde{x}_1$).
 4. Keep the model with the most inliers.
- Finally, refit the best model on all of its inliers.

- ▶ With inlier ratio w , a sample is outlier-free with probability w^s , and all N samples fail with probability $(1 - w^s)^N$. Asking for success probability p :

$$N = \frac{\log(1 - p)}{\log(1 - w^s)}$$

Samples N for $p = 0.99$ (rounded up)

	$s = 4$	$s = 5$	$s = 7$	$s = 8$
	H	E	F	F
$w = 0.5$	72	146	588	1177
$w = 0.3$	567	1893	21 055	70 188

- ▶ N grows exponentially with the sample size s . At 30% inliers the eight-point algorithm needs about 37 times as many samples as the five-point solver (70 188 against 1893).
- ▶ This is why minimal solvers matter: they shrink s inside the exponent.

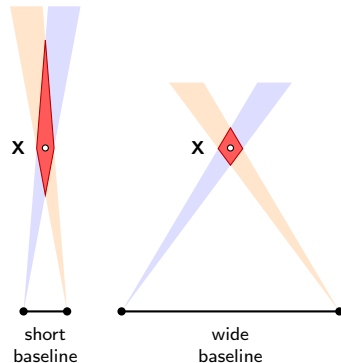
Fischler and Bolles, "Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography," *Communications of the ACM* 24(6), 1981, doi:10.1145/358669.358692, which derives the same number of trials.

- **Four candidates.** Take the SVD $E = U \text{diag}(1, 1, 0) V^T$ with $\det U = \det V = 1$. Then

$$R \in \{UWV^T, UW^T V^T\}, \quad \mathbf{t} = \pm \mathbf{u}_3,$$

with $W = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}$ and $\mathbf{u}_3$ the last column of U .

- **Cheirality** (a point must lie in front of a camera): triangulate a match with each candidate; only one puts it at positive depth in both views.
- **Scale:** E is defined only up to scale (any multiple of E satisfies $\hat{\mathbf{x}}_2^T E \hat{\mathbf{x}}_1 = 0$), so the length of $\mathbf{t}$ is not observable and is set to $\|\mathbf{t}\| = 1$. Metric scale needs outside information, such as a known length.
- **Triangulation (DLT):** $\tilde{\mathbf{x}} \times P\tilde{\mathbf{X}} = \mathbf{0}$ gives two independent equations per view, $u \mathbf{p}^{3\top} \tilde{\mathbf{X}} - \mathbf{p}^{1\top} \tilde{\mathbf{X}} = 0$ and $v \mathbf{p}^{3\top} \tilde{\mathbf{X}} - \mathbf{p}^{2\top} \tilde{\mathbf{X}} = 0$, with $\mathbf{p}^{k\top}$ row k of P . Stack both views into a 4×4 system, take the SVD null vector, then refine $\mathbf{X}$ by minimizing the reprojection error.



Both cameras see $\mathbf{X}$ (white dot) with the same $\pm 3^\circ$ ray error. With nearly parallel rays (short baseline, little parallax) the possible positions (red) stretch about four times as far in depth.

- **Rectified rig:** two cameras with the same K (zero skew, $f_x = f_y$, written f and in pixels from here on) and parallel optical axes, centres C_L and C_R a baseline B apart along x . A point $\mathbf{X} = (X, Y, Z)^\top$ in left-camera coordinates lands on the same row v in both images.

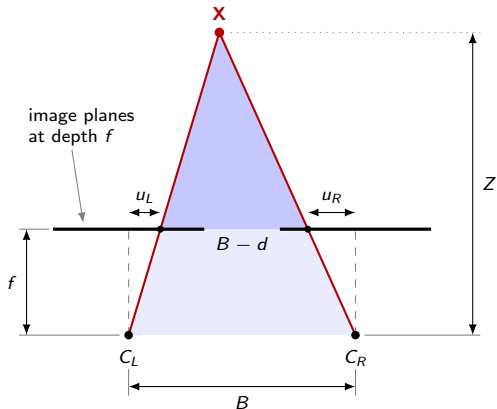
- **Columns,** by the pinhole model (defined above):

$$u_L = \frac{fX}{Z} + c_x, \quad u_R = \frac{f(X - B)}{Z} + c_x.$$

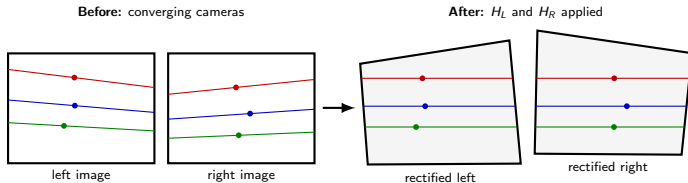
- **Disparity** $d = u_L - u_R$ (pixels); c_x cancels:

$$d = \frac{fB}{Z} \iff Z = \frac{fB}{d} \quad (\text{in the units of } B).$$

- **Similar triangles** (shaded; f , d as lengths on the image plane): apex $\mathbf{X}$, bases B at depth 0 and $B - d$ at depth f , so $(B - d)/(Z - f) = B/Z$, again $Z = fB/d$. Only f/d enters, so pixel units give the same Z .
- Depth is inversely proportional to disparity: near points shift a lot, and $d \rightarrow 0$ as $Z \rightarrow \infty$.



Top view. The rays from C_L and C_R meet at $\mathbf{X}$. u_L and u_R are measured from each principal point ($u_R < 0$ here).



Computed example: three matches and their epipolar lines; after warping, each pair of lines is one row.

- **Goal:** warp both images by homographies H_L , H_R (defined above) that send both epipoles to infinity along x . Matching then becomes a 1D scan along one row.
- **Calibrated method** (after Fusiello et al. 2000, who take $\mathbf{r}_1$ along $C_L - C_R$; the sign here keeps the images upright): keep both centres and rotate both cameras to one orientation $R_n = [\mathbf{r}_1 \ \mathbf{r}_2 \ \mathbf{r}_3]^T$, whose rows are the new camera axes in world coordinates: $\mathbf{r}_1 = (C_R - C_L) / \|C_R - C_L\|$ (the baseline), $\mathbf{r}_2 = \mathbf{k} \times \mathbf{r}_1 / \|\mathbf{k} \times \mathbf{r}_1\|$, $\mathbf{r}_3 = \mathbf{r}_1 \times \mathbf{r}_2$, where $\mathbf{k}$ is the old left optical axis; give both cameras one K_n . Image i is resampled with $H_i = K_n R_n R_i^T K_i^{-1}$ (K_i , R_i : original camera) by bilinear interpolation (a distance-weighted blend of the four input pixels around each source position).
- **Limits:** the construction fails for forward motion along the optical axis, where the epipole lies inside the image. Loop and Zhang (1999) need only the epipolar geometry (F) and choose H_L , H_R to minimise image distortion.

Fusiello, Trucco, and Verri, "A Compact Algorithm for Rectification of Stereo Pairs," *Machine Vision and Applications* 12(1), 2000, [doi:10.1007/s001380050120](https://doi.org/10.1007/s001380050120), Sections 3–4; Loop and Zhang, "Computing Rectifying Homographies for Stereo Vision," CVPR 1999, [doi:10.1109/CVPR.1999.786928](https://doi.org/10.1109/CVPR.1999.786928).

- Differentiating $Z = fB/d$: a matching error of Δd pixels gives

$$\Delta Z \approx \left| \frac{\partial Z}{\partial d} \right| \Delta d = \frac{Z^2}{fB} \Delta d, \quad \frac{\Delta Z}{Z} \approx \frac{Z}{fB} \Delta d.$$

- **KITTI** (car-mounted rig, 2011-09-26 calibration, grayscale pair): rectified $f = 721.54$ px and $fB = 387.57$ pixel-metres, so $B = 0.537$ m; the paper gives both stereo baselines as approximately 54 cm.

Depth Z	Disparity $d = fB/Z$	ΔZ at $\Delta d = 1$ px	$\Delta Z/Z$	Z at $d \pm 1$ px
5 m	77.5 px	0.06 m	1.3%	4.94–5.07 m
10 m	38.8 px	0.26 m	2.6%	9.75–10.26 m
20 m	19.4 px	1.03 m	5.2%	19.02–21.09 m
50 m	7.8 px	6.45 m	12.9%	44.29–57.41 m

- **Levers**: a longer baseline or a longer focal length divides every ΔZ by the same factor; sub-pixel matching shrinks Δd itself (at 0.25 px every ΔZ above drops fourfold).
- **Trade-offs**: doubling B also doubles every disparity. With a search range of $D = 192$ disparities ($d \leq 191$), the nearest measurable depth $fB/(D - 1)$ moves from 2.0 m to 4.1 m, and the region both cameras see starts farther out. A longer f narrows the field of view.

Geiger et al., "Vision Meets Robotics: The KITTI Dataset," *IJRR* 32(11), 2013, [doi:10.1177/0278364913491297](https://doi.org/10.1177/0278364913491297), Section II; raw-data calibration 2011_09_26/calib_cam_to_cam.txt, cameras 0 and 1.

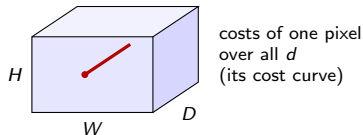
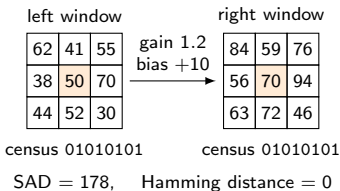
- **Matching cost** $C(\mathbf{p}, d)$: left pixel $\mathbf{p} = (u, v)$ against right pixel $(u - d, v)$ over a window $\mathcal{W}$, e.g. the sum of absolute differences (SAD)

$$C_{\text{SAD}}(\mathbf{p}, d) = \sum_{(i,j) \in \mathcal{W}} |I_L(u+i, v+j) - I_R(u+i-d, v+j)|.$$

Squared differences (SSD) work alike; normalized cross-correlation (NCC) of zero-mean, unit-norm windows cancels gain and bias.

- **Census transform**: one bit per neighbour, 1 when it is darker than the centre; the cost is the Hamming distance (defined above) between the two bit strings. Only comparisons count: gain and bias cancel, and an outlier neighbour flips only its own bit.
- **Cost volume** $C(u, v, d)$, $d \in \{0, \dots, D-1\}$: an $H \times W \times D$ array (the disparity space image of Scharstein and Szeliski). Winner-take-all: $d_p = \arg \min_d C(\mathbf{p}, d)$.
- **Failures**: textureless regions (flat cost curve), repetitive patterns (several minima), occlusions (no true match), specular surfaces (view-dependent appearance).
- **Left-right check**: with disparity maps d_L and d_R of both images, keep $\mathbf{p}$ only if $|d_L(u, v) - d_R(u - d_L(u, v), v)| \leq 1$.

Zabih and Woodfill, "Non-parametric Local Transforms for Computing Visual Correspondence," ECCV 1994, doi:10.1007/BFb0028345, Section 3; Scharstein and Szeliski, "A Taxonomy and Evaluation of Dense Two-Frame Stereo Correspondence Algorithms," IJCV 47(1-3), 2002, doi:10.1023/A:1014573219977; left-right check: Hirschmüller (next slide), Eq. 15.



Census bits in raster order, centre skipped.

- **Energy** of a disparity map, with neighbours $\mathcal{N}_p$ and $[\cdot]$ equal to 1 when true:

$$E(d) = \sum_p C(p, d_p) + \sum_p \sum_{q \in \mathcal{N}_p} (P_1[|d_p - d_q| = 1] + P_2[|d_p - d_q| > 1])$$

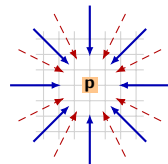
A small P_1 allows slanted and curved surfaces; a larger $P_2 \geq P_1$ charges every jump the same, which keeps depth edges sharp.

- Minimising E over the 2D image is NP-complete for such energies (no known algorithm solves every instance in time polynomial in the image size). Along a 1D path it is exact by **dynamic programming**: each pixel reuses the best costs of the previous pixel $p - r$ along direction r ,

$$L_r(p, d) = C(p, d) + \min (L_r(p-r, d), L_r(p-r, d-1) + P_1, \\ L_r(p-r, d+1) + P_1, \min_i L_r(p-r, i) + P_2) \\ - \min_k L_r(p-r, k).$$

The last term keeps the values bounded without moving the minimum.

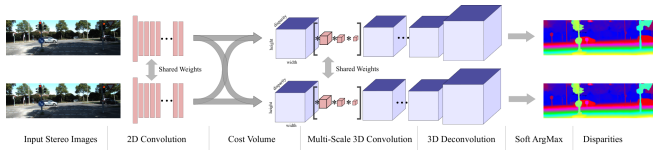
- **Aggregate** $S(p, d) = \sum_r L_r(p, d)$ over at least 8 paths, 16 recommended, then winner-take-all on S . Cost $O(WHD)$.



Paths into p : 8 solid, plus 8 dashed for 16.

- **Speed:** 1–2 s on typical test images (Teddy, 450×375 : 1.8 s on a 2.2 GHz CPU).
- **Deployed:** Daimler ran it on a low-power field-programmable gate array (FPGA) for driver assistance (27 Hz, under 3 W); by 2011 the German Aerospace Center (DLR) had run it on over 100 TB of aerial and satellite images.
- OpenCV ships a variant, StereoSGBM: 5 directions by default, 8 in full mode (docs.opencv.org).

Hirschmüller, "Stereo Processing by Semiglobal Matching and Mutual Information," *IEEE TPAMI* 30(2), 2008, doi:10.1109/TPAMI.2007.1166, Eqs. 11–15; Hirschmüller, "Semi-Global Matching: Motivation, Developments and Applications," Photogrammetric Week 2011, pp. 173–184.



Kendall et al., “End-to-End Learning of Geometry and Context for Deep Stereo Regression,” ICCV 2017, [arXiv:1703.04309v1](https://arxiv.org/abs/1703.04309v1), Figure 1.

- ▶ **GC-Net:** one 2D CNN with shared weights computes features for both images. For each disparity level, the left feature at (u, v) is concatenated with the right feature at $(u - d, v)$: a 4D cost volume (height, width, disparity, feature).
- ▶ **3D convolutions** (covered earlier for video) in an encoder-decoder turn it into one cost c_d per pixel and disparity.
- ▶ **Soft argmin** (“Soft ArgMax” in the figure):

$$\hat{d} = \sum_{d=0}^{D-1} d \cdot \sigma_d(-c)$$
 with σ the softmax over the D costs c of a pixel. It is differentiable and gives sub-pixel disparities; training minimises the L1 error $|d - \hat{d}|$ on pixels with ground truth.
- ▶ **PSMNet:** spatial pyramid pooling averages the features over 64×64 , 32×32 , 16×16 and 8×8 windows, then upsamples and concatenates them, so each pixel carries context at four scales.
- ▶ **Stacked hourglass:** three 3D encoder-decoders in a row, each predicting a disparity map with its own loss.
- ▶ **Metrics:** end-point error (EPE), the mean $|\hat{d} - d|$ in pixels; KITTI 2015 **D1**, the share of pixels whose error is ≥ 3 px and $\geq 5\%$ of the true disparity.
- ▶ GC-Net $\rightarrow$ PSMNet: EPE 2.51 $\rightarrow$ 1.09 px on Scene Flow (rendered scenes); D1 over all pixels 2.87% $\rightarrow$ 2.32% on the KITTI 2015 test set.

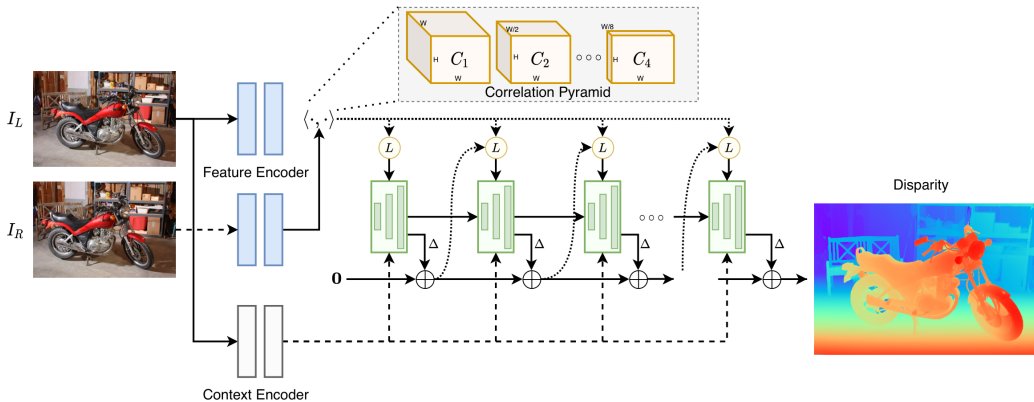
Chang and Chen, “Pyramid Stereo Matching Network,” CVPR 2018, [arXiv:1803.08669v1](https://arxiv.org/abs/1803.08669v1), Sections 3.2–3.4, Tables 4–5; D1: KITTI 2015 stereo benchmark, cvlibs.net/datasets/kitti.

- ▶ **RAFT** (optical flow, covered earlier as a dense motion field): a CNN g maps both frames to features at $1/8$ resolution. The **correlation volume** holds the dot product of every pair of feature vectors, $C_{ijkl} = \sum_h g(l_1)_{ijh} g(l_2)_{klh}$, average-pooled into a 4-level pyramid.
- ▶ **Recurrent updates**: the flow starts at 0. Each iteration, a lookup L_C reads the correlations within radius r of the current estimate at every pyramid level, and a convolutional GRU (the GRU covered earlier, with convolutions in place of its fully connected layers) predicts an increment.
- ▶ **RAFT-Stereo**: matches share a row, so only same-row pairs are kept, $C_{ijk} = \sum_h g(l_L)_{ijh} g(l_R)_{ikh} (H \times W \times W)$, pooled along the last axis only. GRUs at $1/8$, $1/16$ and $1/32$ resolution exchange hidden states, spreading context across textureless regions. No 3D convolutions: megapixel images fit. Loss: L1 on iterate i of N , weighted 0.9^{N-i} .
- ▶ **Zero-shot test** (trained on Scene Flow only, no training on any test dataset), % of pixels above the error threshold. KITTI 2015: driving; Middlebury: indoor, half resolution; ETH3D: indoor and outdoor.

	KITTI 2015, 3 px	Middlebury, 2 px	ETH3D, 1 px
PSMNet	16.3	25.1	23.8
RAFT-Stereo	5.74	12.59	3.28

Table 1. Fine-tuned on Middlebury, RAFT-Stereo ranked first there at submission, its 1 px error 29% below the next method.

Teed and Deng, “RAFT: Recurrent All-Pairs Field Transforms for Optical Flow,” ECCV 2020, [arXiv:2003.12039v3](#); Lipson, Teed, and Deng, “RAFT-Stereo: Multilevel Recurrent Field Transforms for Stereo Matching,” 3DV 2021, [arXiv:2109.07547v1](#).



Lipson, Teed, and Deng, [arXiv:2109.07547v1](https://arxiv.org/abs/2109.07547v1), Figure 1.

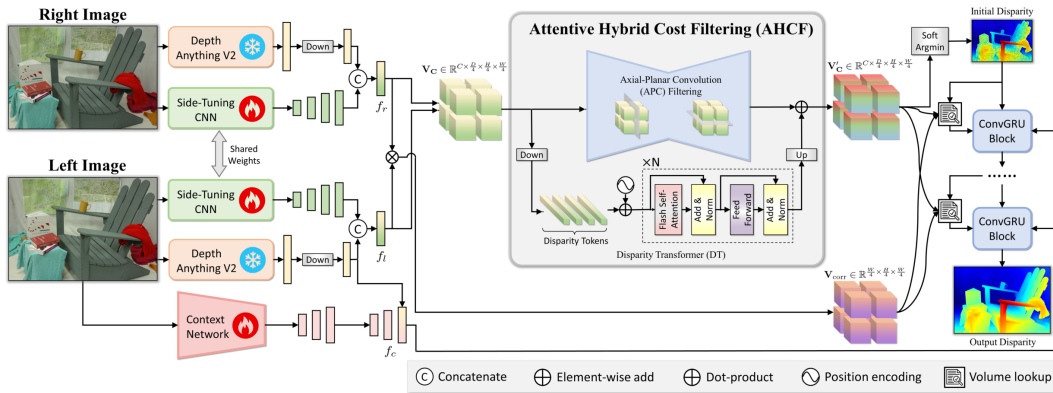
- **Blue:** the feature encoder runs on I_L and I_R ; same-row dot products $\langle \cdot, \cdot \rangle$ build the correlation pyramid $C_1, \dots, C_4$.
White: the context encoder sees I_L only; it sets the GRUs' initial hidden state and feeds every GRU step.
- **Green:** from disparity 0 , each GRU step looks up the pyramid (L) at the current estimate and adds an update Δ ; the final estimate is upsampled to full resolution.

- ▶ **Goal:** one model that works zero-shot in new domains.
- ▶ **Data:** the FoundationStereo dataset (FSD), 1M synthetic stereo pairs; a self-curation loop drops ambiguous pairs (repeated texture, reflections, flat colour) and replaces them with newly generated ones.
- ▶ **Side-Tuning Adapter (STA):** features of a frozen Depth Anything V2 (covered earlier) plus a trainable CNN bring monocular priors learned on real images.
- ▶ **Attentive Hybrid Cost Filtering (AHCF):** a cost volume of group-wise correlations (feature channels split into 8 groups, one dot product per group) and concatenated features, filtered by axial-planar convolutions (a 3D convolution split into spatial and disparity parts) and a Disparity Transformer (self-attention along disparity).
- ▶ A soft argmin gives the initial disparity; convolutional GRU updates refine it as in RAFT-Stereo.

Model	Training data	Middlebury BP-2	ETH3D BP-1
RAFT-Stereo	Scene Flow	12.6	3.3
FoundationStereo	Scene Flow	5.5	1.8
FoundationStereo	FSD + 6 public sets	1.1	0.5

Zero-shot results (Table 2). BP- X : % of pixels with error above X px; no method saw the target datasets. Fine-tuned, FoundationStereo ranked first on the ETH3D and Middlebury leaderboards at submission.

Wen et al., “FoundationStereo: Zero-Shot Stereo Matching,” CVPR 2025, [arXiv:2501.09898v4](https://arxiv.org/abs/2501.09898v4), Sections 3–4.



Wen et al., [arXiv:2501.09898v4](https://arxiv.org/abs/2501.09898v4), Figure 2.

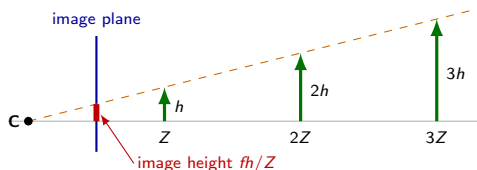
- ▶ **Left:** frozen Depth Anything V2 (snowflake) plus trainable side-tuning CNN (flame) give f_l , f_r ; the context network sees the left image only. **Middle:** AHCF filters the hybrid volume V_C (APC, DT); a soft argmin gives the initial disparity.
- ▶ **Right:** convolutional GRU blocks refine it, looking up the filtered volume and the correlation volume V_{corr} .

- **Structured light:** a projector replaces the second camera. The shift of its known pattern in the camera image is a disparity, so $Z = fb/d$ as in stereo, with projector-camera baseline b .
- **Time of flight (ToF):** light goes to the surface and back, so the distance along the pixel's ray is $\ell = ct/2$ (c : speed of light, t : round-trip time), and the depth is $Z = \ell / \|K^{-1}\tilde{x}\| \leq \ell$ (defined above). ToF cameras read t from the phase $\varphi = 2\pi f_m t$ of light modulated at f_m , which wraps beyond $\ell = c/(2f_m)$; several frequencies are combined. **LiDAR** (light detection and ranging) times laser pulses along scanned beams.

Sensor (example)	Output	Range, accuracy	Weaknesses
Structured light (first Kinect)	640×480 depth, 30 Hz	short: $b = 7.5$ cm	sunlight swamps the pattern; devices interfere
ToF camera (Kinect One)	512×424 depth	driver cuts off near 4.5 m	multipath (light arriving via other surfaces); mixed pixels at depth edges; interference
LiDAR (Velodyne HDL-64E, KITTI)	64 beams, 10 Hz; about 120K points per scan	120 m, 2 cm accuracy	sparse; fog, rain, snow; sunlight and other LiDARs interfere; cost, moving parts
Passive stereo (KITTI rig)	every pixel, 1242×375	error grows as $Z^2/(fB)$	textureless and repetitive regions, occlusions, darkness

- **Passive stereo wins** on cost, resolution and outdoor use: cameras emit nothing, so sunlight and other vehicles do not interfere, and range scales with B and f . Active sensors win on blank walls, in the dark, and (LiDAR) at long range.

Sarbolandi, Lefloch, and Kolb, "Kinect Range Sensing: Structured-Light versus Time-of-Flight Kinect," *CVIU* 139, 2015, [arXiv:1505.05459v1](#), Sections 1–2, Table 2; Li and Ibanez-Guzman, "Lidar for Autonomous Driving: The Principles, Challenges, and Trends for Automotive Lidar and Perception Systems," *IEEE Signal Processing Magazine* 37(4), 2020, [arXiv:2004.08467v1](#), Eq. 2; HDL-64E: Geiger et al. 2013, Sections II and III-A.



Camera centre C , image plane at focal length f . Objects of height h , $2h$, $3h$ at depths Z , $2Z$, $3Z$ give the same image, because the image height is fh/Z .

- ▶ **Ill-posed:** back-projection gives only a ray, $\mathbf{X}_c = Z K^{-1} \tilde{\mathbf{x}}$, and scaling the whole scene by any $\alpha > 0$ leaves every pixel unchanged (both defined above), so one image cannot fix Z .
- ▶ **Monocular cues:** perspective (parallel lines converge), relative and familiar size, texture gradient (a uniform texture looks denser farther away), occlusion, shading, defocus blur, and height in the image (on a ground plane, farther points lie closer to the horizon).
- ▶ **Learned reliably:** depth ordering (what occludes what) and scene shape up to a global scale.
- ▶ **Absolute scale:** only through priors on familiar object sizes and on the camera; a network trained on one camera inherits its scale.
- ▶ Most of the scenes that fit an image are physically implausible, yet “At least one major ambiguity remains, though: the global scale.”

Quotation: Eigen et al., NIPS 2014, [arXiv:1406.2283v1](https://arxiv.org/abs/1406.2283v1), Section 1.

- ▶ **Two networks:** a coarse network (five convolution and pooling layers, then two fully connected layers) predicts the depth layout from the whole image; a fine network refines it locally, taking the coarse map as an extra feature map. The output is log depth.
- ▶ **Scale-invariant loss** with $e_i = \ln y_i - \ln y_i^*$:

$$L = \frac{1}{n} \sum_i e_i^2 - \frac{\lambda}{n^2} \left(\sum_i e_i \right)^2, \quad \lambda \in [0, 1]$$

y_i is the predicted and y_i^* the ground-truth depth at pixel i ; n counts the pixels that have ground truth.

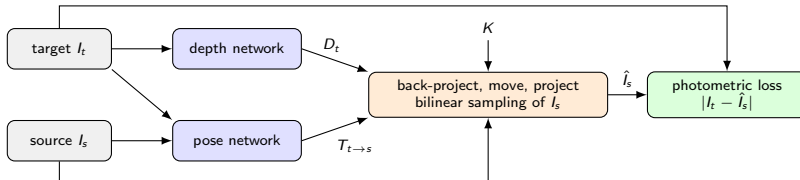
- ▶ $\lambda = 0$ is the per-pixel ℓ_2 error in log space. $\lambda = 1$ is exactly scale-invariant: it equals that error after the best global rescaling of y . Training used $\lambda = 0.5$, which kept good absolute scale.

Worked example. Predict $y_i = 2 y_i^*$ at every pixel. Then $e_i = \ln 2 \approx 0.69$ for all i , and

$$L = (1 - \lambda)(\ln 2)^2 = \begin{cases} 0.48 & \lambda = 0 \\ 0.24 & \lambda = 0.5 \\ 0 & \lambda = 1 \end{cases}$$

- ▶ **Labels:** NYU Depth v2 records 464 indoor scenes as Microsoft Kinect video; KITTI projects a car-mounted rotating LiDAR into the camera.
- ▶ **Sparse and noisy:** reprojected LiDAR covers under 5% of KITTI pixels, and Kinect depth has holes at windows, specular surfaces, and object boundaries. Pixels without depth are masked out of the loss.

Eigen et al., NIPS 2014, [arXiv:1406.2283v1](#), Sections 3–4, Eqs. 3–4; LiDAR coverage: Godard et al., CVPR 2017, [arXiv:1609.03677v3](#), Section 4.



Training graph of Zhou et al. in this lecture's notation ($I_s: I_{t\pm 1}$). To predict depth at test time, only the depth network runs.

- **Warp:** back-project pixel $\mathbf{p}_t$ with its predicted depth, move it by $T_{t \rightarrow s}$ (rotation R , translation $\mathbf{t}$), and project it into the source:

$$\tilde{\mathbf{p}}_s \sim K(R D_t(\mathbf{p}_t) K^{-1} \tilde{\mathbf{p}}_t + \mathbf{t})$$

- **Bilinear sampling** (defined above) reads I_s at the non-integer point $\mathbf{p}_s$ to give $\hat{I}_s(\mathbf{p}_t)$, and is differentiable in D_t and $T_{t \rightarrow s}$.

- **No depth labels:** the photometric loss trains both networks. It assumes a static scene, no occlusion, and Lambertian surfaces (brightness independent of viewpoint).
- **Scale:** the scale ambiguity (defined above) remains: D_t and $\mathbf{t}$ can grow together without moving $\tilde{\mathbf{p}}_s$, so video gives depth only up to scale.
- **Stereo pairs** (Godard et al., 2017): the source is the other rectified camera, and the known baseline B gives metric depth via $Z = fB/d$ (defined above).

Zhou et al., "Unsupervised Learning of Depth and Ego-Motion from Video," CVPR 2017, [arXiv:1704.07813v2](https://arxiv.org/abs/1704.07813v2), Sections 3.1–3.3;
Godard et al., [arXiv:1609.03677v3](https://arxiv.org/abs/1609.03677v3), Section 3.

- **Photometric error** between the target and a warped source, with $\alpha = 0.85$:

$$pe(I_a, I_b) = \frac{\alpha}{2} (1 - \text{SSIM}(I_a, I_b)) + (1 - \alpha) \|I_a - I_b\|_1$$

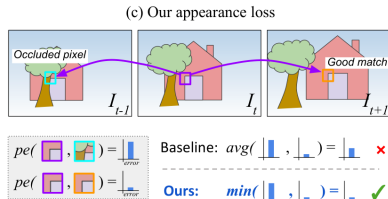
- **SSIM** (structural similarity) compares patches x and y in a small window:

$$\text{SSIM}(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}$$

μ , σ^2 : local means and variances; σ_{xy} : covariance;
 C_1 , C_2 : small constants. SSIM is 1 only for identical patches.

- **Per-pixel minimum:** $L_p = \min_s pe(I_t, \hat{I}_s)$, so a pixel occluded in one source is matched in the other.
- **Auto-masking:** keep a pixel only if $\min_s pe(I_t, \hat{I}_s) < \min_s pe(I_t, I_s)$, which drops static-camera frames and objects moving with the camera (otherwise holes of infinite depth).
- **Full-resolution multi-scale:** every decoder scale's depth is upsampled to the input resolution before warping, which reduces holes and texture-copy artifacts (image detail wrongly copied into the depth map).

SSIM: Wang et al., IEEE TIP 2004, [doi:10.1109/TIP.2003.819861](https://doi.org/10.1109/TIP.2003.819861), Eq. 13.

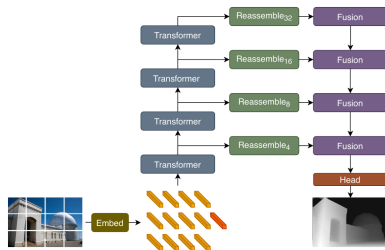


Godard et al., "Digging Into Self-Supervised Monocular Depth Estimation," ICCV 2019, [arXiv:1806.01260v4](https://arxiv.org/abs/1806.01260v4), Figure 3(c); losses from Section 3.

- ▶ **Mixed labels:** metric (laser, calibrated stereo), up to scale (multi-view reconstruction), or disparity up to scale and shift (uncalibrated stereo, 3D films).
- ▶ **MiDaS** predicts disparity d_i (inverse depth) and fits a scale s and shift t to the ground truth d_i^* by least squares before scoring its M valid pixels:

$$\mathcal{L}_{\text{ssi}} = \min_{s,t} \frac{1}{2M} \sum_{i=1}^M (s d_i + t - d_i^*)^2$$

Training instead centres both maps on their medians, scales each by its mean absolute deviation, and drops the largest 20% of residuals from an $|x|$ loss.



Ranftl et al., "Vision Transformers for Dense Prediction," ICCV 2021, [arXiv:2103.13413v1](https://arxiv.org/abs/2103.13413v1), Figure 1 (left); training data: Section 4.1.

- ▶ **MiDaS mixes five datasets** Pareto-optimally (no dataset's loss can fall without another's rising); zero-shot tests on six unseen datasets.
- ▶ **DPT** (dense prediction transformer, right): tokens from four ViT stages (covered earlier) are reassembled into maps at 1/32 to 1/4 resolution and fused by a convolutional decoder; MiDaS loss, about 1.4M images.
- ▶ **Depth Anything** (covered earlier) scaled this recipe with pseudo-labels on 62M unlabeled images.

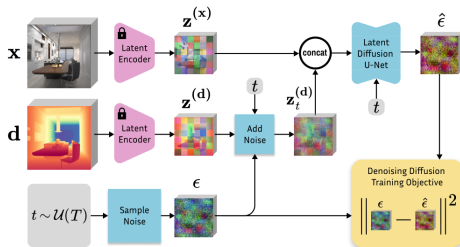
Ranftl et al., TPAMI 2022, [arXiv:1907.01341v3](https://arxiv.org/abs/1907.01341v3), Sections 3–6; Yang et al., [arXiv:2401.10891v2](https://arxiv.org/abs/2401.10891v2), Section 3.

- ▶ **The camera sets the scale:** an object's image height fh/Z (above) is the same with focal length $2f$ at depth $2Z$, so without f the image size cannot separate the two depths.
- ▶ **Metric3D** maps each training image to a canonical camera with focal length f^c , scaling the depth label (or resizing the image) by f^c/f and inverting this at test time; 8M images from 11 datasets (v2: surface normals, 16M images).
- ▶ **UniDepth** predicts a dense map of camera ray angles (azimuth, elevation) that conditions the depth module, and outputs a 3D point per pixel.
- ▶ **Depth Pro** predicts a canonical inverse depth C and returns $D_m = f_{px}/(wC)$ (f_{px} : focal length in pixels, w : image width); a separate head predicts the field of view. A 2.25-megapixel map takes 0.3 s on a V100.

Method	Source of metric scale	Camera at test time
Supervised, one dataset	metric labels from one sensor and camera	same camera as training
Stereo self-supervision	baseline B and focal length f of the training rig	same camera as training
Metric3D, Metric3D v2	canonical camera: labels or images rescaled by f^c/f	focal length required
UniDepth	predicted per-pixel camera rays	none (optional input)
Depth Pro	canonical inverse depth and a predicted focal length	none

Yin et al., ICCV 2023, [arXiv:2307.10984v1](#), Sections 1, 3.1–3.2, 4; Hu et al., TPAMI 2024, [arXiv:2404.15506v4](#); Piccinelli et al., CVPR 2024, [arXiv:2403.18913v1](#); Bochkovskii et al., ICLR 2025, [arXiv:2410.02073v2](#), Sections 1, 3.2–3.3.

- ▶ **Marigold** turns Stable Diffusion v2 (covered earlier) into a depth estimator. The frozen VAE encodes the image $\mathbf{x}$ and the depth map $\mathbf{d}$ (copied to three channels).
- ▶ The U-Net input is the image latent concatenated with the noisy depth latent; only the U-Net is fine-tuned (usual denoising objective, no text prompt).
- ▶ **Synthetic data only:** about 54K Hypersim (indoor) and 20K Virtual KITTI (street) samples with dense, noise-free depth, each normalised by its 2nd and 98th percentiles: the output is affine-invariant (fixed only up to an unknown scale and shift).
- ▶ **Inference:** 50 DDIM steps (covered earlier) from noise, then the VAE decoder. Ten runs from different noise are aligned in scale and shift and merged by the pixelwise median, which cuts the absolute relative error (the mean of $|y_i - y_i^*|/y_i^*$) on NYU Depth v2 by 8%.
- ▶ **Cost:** about 2.5 days of fine-tuning on one RTX 4090 GPU.



“Overview of the Marigold fine-tuning protocol.” Ke et al., “Repurposing Diffusion-Based Image Generators for Monocular Depth Estimation,” CVPR 2024, [arXiv:2312.02145v2](https://arxiv.org/abs/2312.02145v2), Figure 2; numbers from Sections 3–4.

- AbsRel is the absolute relative error defined above; over the N pixels with ground-truth depth y_i^* and predicted depth y_i ,

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_i (y_i - y_i^*)^2}$$

δ_1 : fraction of pixels with $\max(y_i/y_i^*, y_i^*/y_i) < 1.25$ (δ_2, δ_3 : $1.25^2, 1.25^3$); higher is better.

- **NYU Depth v2**: 654 indoor test images from a Kinect, evaluated up to 10 m.
- **KITTI Eigen split**: 697 test images from 29 scenes, reprojected LiDAR, a fixed crop, depth capped at 80 m. Marigold uses a 652-image version; 652 of the 697 frames have improved ground truth.

Alignment uses the ground truth, so compare scores only within one protocol:

Protocol	Used for	Alignment before scoring
Median scaling	self-supervised monocular models	$y \leftarrow y \cdot \text{median}(y^*) / \text{median}(y)$ per image
Least-squares scale and shift	affine-invariant models (MiDaS, DPT, Marigold)	fit s, t on valid pixels (in disparity for disparity models)
No alignment	metric models (Metric3D, UniDepth, Depth Pro)	none: the prediction is scored as it is

Metrics: Eigen et al., [arXiv:1406.2283v1](#), Section 4.3. Protocols: Zhou et al., [arXiv:1704.07813v2](#), Section 4.1; Godard et al., [arXiv:1806.01260v4](#), Section 4.1 and supplementary D.1 (improved ground truth); Ranftl et al., [arXiv:1907.01341v3](#), Section 6. Splits: Godard et al., [arXiv:1609.03677v3](#), Section 4; Ke et al., [arXiv:2312.02145v2](#), Section 4.2.

- ▶ **Camera pose:** the rigid transform $(R, \mathbf{t}) \in SE(3)$ defined above, $\mathbf{X}_c = R \mathbf{X}_w + \mathbf{t}$: six DoF, three for rotation and three for translation.
- ▶ **Relative pose** links two cameras (from E , with $\mathbf{t}$ only up to scale, defined above). **Absolute pose** places one camera in a fixed map frame; this section estimates it.
- ▶ Human pose estimation (body keypoints, covered earlier) is a different task.
- ▶ **Quaternion:**
 $q = (q_w, q_x, q_y, q_z) = q_w + q_x i + q_y j + q_z k$
 with $i^2 = j^2 = k^2 = ijk = -1$. The rotation by angle θ about the unit axis $\mathbf{n}$ is the unit quaternion $q = (\cos \frac{\theta}{2}, \mathbf{n} \sin \frac{\theta}{2})$.
- ▶ **Why networks regress quaternions:** any nonzero 4-vector becomes a valid rotation after division by its norm, which is simpler than projecting a 3×3 output onto $SO(3)$.

Rotation as	Count	Constraint and catch
Matrix R	9	$R^\top R = I$, $\det R = +1$: six constraints, so a raw network output is not a rotation
Euler angles	3	three turns about coordinate axes, e.g. yaw, pitch, roll (vertical, sideways, forward axis); gimbal lock: at pitch $\pm 90^\circ$ the yaw and roll axes align and one DoF is lost
Axis-angle $\theta \mathbf{n}$	3	minimal, but $\pi \mathbf{n}$ and $-\pi \mathbf{n}$ are the same rotation
Unit quaternion q	4	one constraint, $\ q\ = 1$; no singularities; q and $-q$ are the same rotation

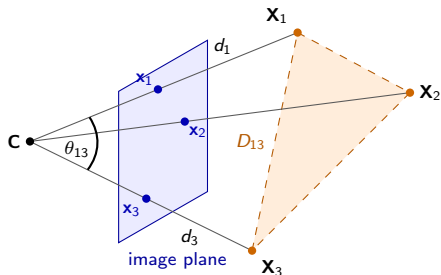
- ▶ **The catch:** $\theta \rightarrow \theta + 2\pi$ flips the sign of q , so every rotation has two labels. Either make labels unique, e.g. store each with $q_w > 0$, plus a sign rule for half-turns ($q_w = 0$), or use a loss that treats q and $-q$ as equal, e.g. $\min(\|q^* - q\|, \|q^* + q\|)$ for the label q^* .

- **Problem:** given K and n matches $\mathbf{x}_i \leftrightarrow \mathbf{X}_i$ from pixels to known 3D map points, find $R, \mathbf{t}$ with $\lambda_i \tilde{\mathbf{x}}_i = K [R | \mathbf{t}] \tilde{\mathbf{X}}_i$. Six unknowns and two equations per match: three matches are the minimum.
- **P3P** (three matches): the unknowns are the distances $d_i = \|\mathbf{X}_i - \mathbf{C}\|$. The map gives $D_{ij} = \|\mathbf{X}_i - \mathbf{X}_j\|$; K gives the angle θ_{ij} between the rays $\hat{\mathbf{x}}_i, \hat{\mathbf{x}}_j$, where $\hat{\mathbf{x}} = K^{-1} \tilde{\mathbf{x}}$. The law of cosines,

$$D_{ij}^2 = d_i^2 + d_j^2 - 2 d_i d_j \cos \theta_{ij}, \quad i < j \in \{1, 2, 3\},$$

has at most four solutions with positive d_i ; a fourth point or the RANSAC inlier count picks one.

- **EPnP** ($n \geq 4$): $\mathbf{X}_i = \sum_{j=1}^4 \alpha_{ij} \mathbf{c}_j$ with four virtual control points $\mathbf{c}_j$. Their 12 camera-frame coordinates come from the eigenvectors of a 12×12 matrix, in $O(n)$ time.
- **In practice:** a minimal solver inside RANSAC (defined above), then reprojection-error refinement on the inliers. At inlier ratio $w = 0.5$ and confidence $p = 0.99$, RANSAC needs 35 samples of size $s = 3$, or 72 of size $s = 4$.

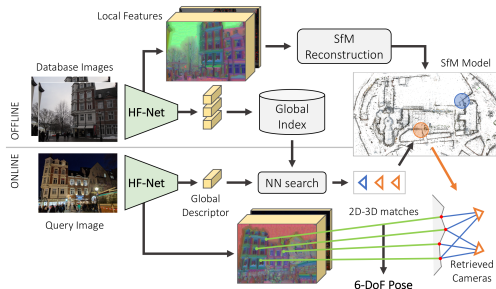


Each ray from the camera centre $\mathbf{C}$ through an image point $\mathbf{x}_i$ meets its map point $\mathbf{X}_i$. The triangle $\mathbf{C}\mathbf{X}_1\mathbf{X}_3$ gives the P3P equation for $(i, j) = (1, 3)$.

Solution count: Gao et al., "Complete Solution Classification for the Perspective-Three-Point Problem," IEEE TPAMI 25(8), 2003,

[doi:10.1109/TPAMI.2003.1217599](https://doi.org/10.1109/TPAMI.2003.1217599). Lepetit, Moreno-Noguer and Fua, "EPnP: An Accurate $O(n)$ Solution to the PnP Problem," IJCV 81(2), 2009, [doi:10.1007/s11263-008-0152-6](https://doi.org/10.1007/s11263-008-0152-6).

- **Map, built offline** by structure from motion (SfM, next slides): images with known poses, their local features, 3D points, and one global descriptor per image.
- **Retrieve:** the query's global descriptor finds the k nearest map images (image retrieval, covered earlier), grouped into places by the 3D points they share.
- **Match:** query keypoints are matched only to the 3D points of each place.
- **Solve:** PnP inside RANSAC gives the 6-DoF pose; the search stops at the first place with a valid pose.
- **Uses:** relocalizing an AR device in a pre-mapped room, or a robot in a prebuilt map.



Sarlin et al., "From Coarse to Fine: Robust Hierarchical Localization at Large Scale," CVPR 2019, [arXiv:1812.03506v2](#), Figure 2; numbers from Table 3 and Sections 5.3–5.4.

- **Retrieval alone is coarse:** for the 98 night-time queries of Aachen Day-Night (map: 4,328 daytime images of a European old town), the top retrieved image's pose is within 0.5 m and 2° for 0.0%; retrieve, match and solve reaches 40.8% (NetVLAD global descriptor, SuperPoint features).
- HF-Net computes the global descriptor and local features in one network; the pipeline runs at 20 frames per second.

- **PoseNet:** a CNN regresses the camera position $\mathbf{C}$ and a quaternion q from one image in 5 ms, trained on poses from SfM:

$$\mathcal{L} = \|\mathbf{C}^* - \mathbf{C}\|_2 + \beta \left\| q^* - \frac{q}{\|q\|} \right\|_2$$

$\mathbf{C}, q$: outputs; $\mathbf{C}^*, q^*$: ground truth; β balances metres against quaternion units (grid search: 120–750 indoors, 250–2000 outdoors).

- **The limit:** the last layer is linear, so the output $\mathbf{b} + \sum_j \alpha_j(I) \mathbf{p}_j$ is a weighted sum of learned base poses $\mathbf{p}_j$; trained on cameras along a line, it predicts poses on that line. Pose regression is “more closely related to pose approximation via image retrieval than to accurate pose estimation via 3D structure.”
- **Scene coordinate regression:** predict the 3D scene point seen at each pixel (or patch), then run PnP inside RANSAC. DSAC* trains end to end through a differentiable RANSAC; ACE trains only an MLP head on a fixed backbone: 291 s of mapping against 15 h for DSAC* on one V100 GPU, a 4 MB map, and accuracy on par.

King's College (outdoor, 140 × 40 m, Cambridge Landmarks), median error

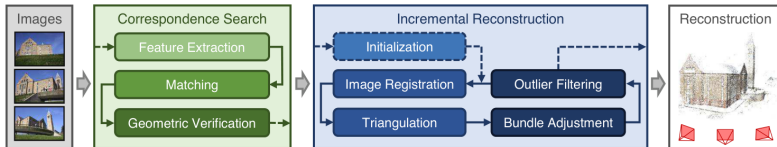
Approach	Method	m / deg
Pose regression	PoseNet	1.92 / 5.40
Image retrieval	DenseVLAD	2.80 / 5.72
	+ interpolation	1.48 / 4.45
2D–3D matching	Active Search	0.42 / 0.55
Scene coordinates	DSAC++	0.18 / 0.3

+ interpolation: poses of the top 20 retrieved images, weighted to best reproduce the query's descriptor.

Sattler et al., “Understanding the Limitations of CNN-based Absolute Camera Pose Regression,” CVPR 2019, [arXiv:1903.07504v1](#), Table 2, Eq. 3, Section 5.

Kendall et al., “PoseNet: A Convolutional Network for Real-Time 6-DOF Camera Relocalization,” ICCV 2015, [arXiv:1505.07427v4](#), Eq. 2, Figure 6. Brachmann and Rother, “Visual Camera Re-Localization from RGB and RGB-D Images Using DSAC,” TPAMI 2021, [arXiv:2002.12324v4](#). Brachmann et al., “Accelerated Coordinate Encoding: Learning to Relocalize in Minutes using RGB and Poses,” CVPR 2023, [arXiv:2305.14059v1](#), Table 3.

Structure from Motion: The Incremental Pipeline



Schönberger and Frahm, "Structure-from-Motion Revisited," CVPR 2016, [doi:10.1109/CVPR.2016.445](https://doi.org/10.1109/CVPR.2016.445), Figure 2 (COLMAP).

- ▶ **Input:** unordered photos of a scene. **Output:** intrinsics K_i , poses $(R_i, \mathbf{t}_i)$, and a sparse point cloud of 3D points $\mathbf{X}_j$, each point with the pixels $\mathbf{x}_{ij}$ that observe it.
- ▶ **Correspondence search:** features (SIFT or learned), matching, and geometric verification with RANSAC on E , F or H (defined above). Verified pairs form the scene graph: images are nodes, pairs are edges.
- ▶ **Initialization:** relative pose and triangulation on a carefully chosen pair in a densely connected part of the graph, because "the reconstruction may never recover from a bad initialization."
- ▶ **Registration:** the next image is located by PnP inside RANSAC from its 2D–3D matches to points already in the model.
- ▶ **Triangulation** adds the new points it sees; **bundle adjustment** (next slide) refines cameras and points jointly; **outlier filtering** drops observations with large reprojection error.
- ▶ "Without further refinement, SfM usually drifts quickly to a non-recoverable state." COLMAP, the open-source system of this paper, adjusts locally after every registration and globally each time the model has grown by a set percentage.

- **Objective:** refine all cameras and points jointly,

$$\min_{\{K_i, R_i, \mathbf{t}_i\}, \{\mathbf{x}_j\}} \sum_{(i,j) \in \mathcal{V}} \rho(\|\mathbf{x}_{ij} - \pi(K_i, R_i, \mathbf{t}_i, \mathbf{x}_j)\|^2)$$

$\mathcal{V}$: observed (image, point) pairs. ρ : a robust loss of the squared residual e : Huber, e for $e \leq \delta^2$ and $2\delta\sqrt{e} - \delta^2$ above, or Cauchy, $\delta^2 \log(1 + e/\delta^2)$ (COLMAP uses it in local adjustment). Both grow slower than e , so a wrong match pulls less.

- **Levenberg–Marquardt (LM):** residuals $\mathbf{r}(\theta)$ over all parameters θ , Jacobian $J = \partial \mathbf{r} / \partial \theta$. Each step solves $(J^\top J + \mu I) \Delta \theta = -J^\top \mathbf{r}$. If the cost drops, accept and shrink μ (toward Gauss–Newton, the exact step for linearized $\mathbf{r}$); if not, grow μ ($\Delta \theta \approx -J^\top \mathbf{r} / \mu$, a short gradient-descent step).

- **Gauge freedom:** a similarity transform of the whole scene ($3 + 3 + 1 = 7$ DoF) changes no residual, so $J^\top J$ is singular: fix the first camera and the scale.

Triggs, McLauchlan, Hartley and Fitzgibbon, “Bundle Adjustment: A Modern Synthesis,” Vision Algorithms: Theory and Practice, LNCS 1883, 2000, doi:10.1007/3-540-44480-7_21, Sections 6.1 and 9.

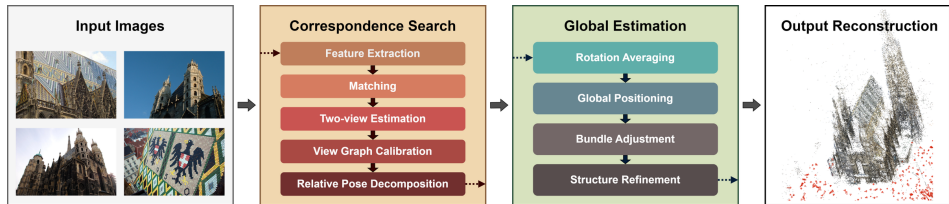
	cameras i			points j			
obs.	1	2	3	1	2	3	4
(1, 1)	■			■			
(1, 2)	■				■		
(1, 3)	■					■	
(2, 1)		■		■			
(2, 2)		■			■		
(2, 4)		■					■
(3, 2)			■		■		
(3, 3)			■			■	
(3, 4)			■				■

Sparsity of J : each residual involves one camera and one point.

Schur complement: split $J^\top J + \mu I$ into U (cameras), V (points) and W (coupling), and $-J^\top \mathbf{r}$ into $\mathbf{g}_c$, $\mathbf{g}_p$. V is block diagonal (3×3 per point), so V^{-1} is cheap. Eliminating the points leaves the reduced camera system

$$(U - WV^{-1}W^\top) \Delta \theta_c = \mathbf{g}_c - WV^{-1}\mathbf{g}_p,$$

small because cameras are far fewer than points; $U - WV^{-1}W^\top$ is the Schur complement of V . Then $\Delta \theta_p = V^{-1}(\mathbf{g}_p - W^\top \Delta \theta_c)$.



Pan et al., “Global Structure-from-Motion Revisited,” ECCV 2024, [arXiv:2407.20219v2](https://arxiv.org/abs/2407.20219v2), Figure 2, Sections 1, 2.2, Eqs. 3–4.

- ▶ **Global SfM** solves all cameras at once. **Rotation averaging** fits every R_i to the pairwise rotations of the scene graph, $R_{ij} \approx R_j R_i^T$, with outliers down-weighted. Positions, triangulation and bundle adjustment follow.
- ▶ Positions usually come from **translation averaging** of the pairwise directions $\mathbf{t}_{ij}$, which lack scale and degenerate under nearly collinear motion.

- ▶ **GLOMAP** replaces translation averaging and triangulation with one global positioning of camera centres $\mathbf{C}_i$ and points $\mathbf{X}_k$:

$$\min_{\mathbf{X}, \mathbf{C}, s} \sum_{i,k} \rho(\|\mathbf{v}_{ik} - s_{ik}(\mathbf{X}_k - \mathbf{C}_i)\|_2), \quad s_{ik} \geq 0$$

$\mathbf{v}_{ik}$: unit world-frame ray direction from camera i to point k ; s_{ik} : a free scale. With the best s_{ik} the error is the sine of the angle between the two rays, capped at 1, so outliers stay bounded; random initialization converges.

	Incremental (COLMAP)	Global (GLOMAP)
Speed	bundle adjustment repeated as the model grows	adjustment rounds only at the end
Robustness	each new image must pass PnP with RANSAC; a bad seed pair can ruin the run	fails when rotation averaging fails, e.g. in symmetric scenes
Drift	accumulates along the registration order	all pairwise constraints are used at once
Typical use	the standard general-purpose tool	large collections and long sequences

Data	Measure	GLOMAP	COLMAP	Theia
ETH3D SLAM	recall, 0.1 m (%)	66.4	57.9	62.8
	time (s)	133.5	1115.4	91.8
LaMAR	recall, 1 m (%)	49.1	32.0	11.0
	time (s)	12,405	354,660	1,542

- **Recall:** % of cameras whose position, after aligning the reconstruction to the ground truth, is within the threshold; averaged over scenes.
- **Times** exclude feature extraction and matching, which all methods share.
- **ETH3D SLAM:** video sequences with sparse features, moving objects and strong lighting changes. **LaMAR:** tens of thousands of AR-device images per scene. **Theia:** another global system.

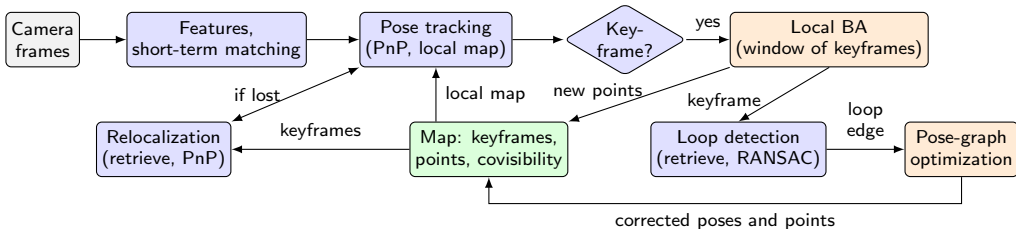
Pan et al., [arXiv:2407.20219v2](#), Sections 1, 2.5, 3.3, 4 and 4.4, Tables 1 and 4. Incremental column: Schönberger and Frahm, CVPR 2016, Sections 2.2 and 4.4.

- ▶ **SLAM, stated formally** (Cadena et al.): “SLAM comprises the simultaneous estimation of the state of a robot equipped with on-board sensors, and the construction of a model (the map) of the environment that the sensors are perceiving.” In visual SLAM the state is the camera pose $T_t \in SE(3)$ at each time t .
- ▶ **Chicken and egg**: tracking the pose needs a map to register each frame against, and building the map needs the poses that triangulate it, so both are estimated together.
- ▶ **Online and causal**: the estimate at time t uses only frames up to t and must keep pace with the camera.

	Visual odometry (VO)	Visual SLAM	SfM (defined above)
Input	ordered video stream	ordered video stream	unordered image set
Processing	online, causal	online, causal	offline, all images available from the start
Revisited places	not recognized: the world looks like an “infinite corridor”	recognized by loop closure, so the map keeps the true topology	matched like any other overlapping pair
Error over time	drift grows without bound	reset at each loop closure	no time order; incremental SfM drifts along the registration order, which global bundle adjustment (BA) corrects

Cadena et al., “Past, Present, and Future of Simultaneous Localization And Mapping: Towards the Robust-Perception Age,” IEEE T-RO 2016, [arXiv:1606.05830v4](#), Sec. I.

- “The front-end abstracts sensor data into models that are amenable for estimation, while the back-end performs inference on the abstracted data produced by the front-end” (Cadena et al., Sec. II).



Blue: front-end (data association); orange: back-end (estimation); green: the map that both read and write.

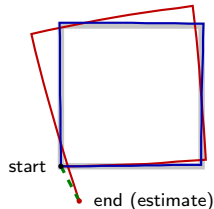
- **Front-end:** data association (deciding which measurements show the same 3D point): extract features (ORB, defined above), match them to the local map, and estimate the pose from these 2D–3D matches (PnP, defined above), refined by motion-only BA (points fixed).
- **Keyframes:** only frames that add new views enter the back-end. BA cost grows with the number of keyframes, so redundant ones are culled.
- **Back-end:** local BA (defined above) over nearby keyframes (covisible, i.e. sharing map points, or recent); after a loop closure, a pose graph: all keyframe poses, linked by relative-pose edges.
- **Relocalization:** when tracking fails, retrieve similar keyframes and solve PnP in RANSAC (defined above); loop detection is the same long-term data association.

- **Drift:** each relative pose carries a small error, and chaining them makes the error grow without bound. One camera cannot observe scale, so the map drifts in all 7 gauge DoF (defined above): rotation, translation and scale.
- **Detect the revisit** (place recognition): image retrieval with global descriptors and approximate nearest-neighbour search (covered earlier), or a **bag of binary words**. DBoW2 quantizes BRIEF descriptors (the descriptor inside ORB) with a vocabulary tree (hierarchical k-medians clustering, 10 branches, 6 levels; its 10^6 leaves are the words), so an image becomes a histogram of words, weighted by tf-idf (term frequency times inverse document frequency: words rare across images count more). An inverted index (each word's list of images) returns the candidates.
- **Verify:** RANSAC (defined above) on a fundamental matrix with at least 12 inliers (DBoW2), or on a 3D-to-3D similarity (ORB-SLAM, next slide).
- **Correct:** add the loop edge and optimize a pose graph over similarities

$$S = \begin{bmatrix} sR & t \\ 0 & 1 \end{bmatrix} \in \text{Sim}(3) \text{ with scale } s > 0:$$

$$\min_{\{S_i\}} \sum_{(i,j) \in \mathcal{E}} \mathbf{e}_{ij}^\top \Lambda_{ij} \mathbf{e}_{ij}, \quad \mathbf{e}_{ij} = \log_{\text{Sim}(3)}(S_{ij} S_i^{-1}) \in \mathbb{R}^7$$

S_i : world-to-keyframe similarity; S_{ij} : measured relative similarity (odometry or loop edge); $\log$: the 7-vector of rotation, translation and log-scale; Λ_{ij} : information matrix, the inverse covariance of the edge (identity in ORB-SLAM); one keyframe is held fixed to remove the 7 gauge DoF.



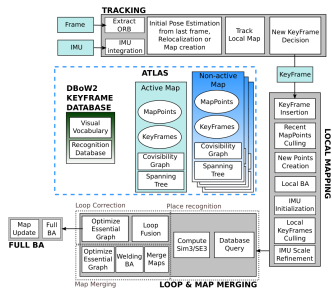
— ground truth — estimate
 - - loop edge — after pose graph

Computed for this slide: a square loop with a heading bias and 0.6% scale drift per step, corrected by a 2D similarity pose graph with one loop edge.

Strasdat et al., "Scale Drift-Aware Large Scale Monocular SLAM," RSS 2010, [doi:10.15607/RSS.2010.VI.010](https://doi.org/10.15607/RSS.2010.VI.010).

Gálvez-López and Tardós, "Bags of Binary Words for Fast Place Recognition in Image Sequences," IEEE T-RO 2012, [doi:10.1109/TRO.2012.2197158](https://doi.org/10.1109/TRO.2012.2197158).

- ▶ **ORB-SLAM** (Mur-Artal et al., T-RO 2015): monocular and keyframe-based; the same ORB features (defined above) serve tracking, mapping, relocalization and loop closing.
- ▶ **Three parallel threads.** Tracking: match the previous frame, motion-only BA, match the local map, decide on keyframes. Local mapping: triangulate new points, local BA, cull points and redundant keyframes. Loop closing: DBoW2 candidates, a Sim(3) found by RANSAC, fusion of duplicate points, essential-graph optimization.
- ▶ **Covisibility graph:** a node per keyframe and an edge when two keyframes share at least 15 map points. **Essential graph:** a spanning tree (the fewest covisibility edges that still connect all keyframes), the covisibility edges with at least 100 shared points, and the loop edges; sparse enough for real-time pose-graph optimization.
- ▶ **ORB-SLAM2** (T-RO 2017) adds stereo and RGB-D (colour plus depth) input, so the map has metric scale.
- ▶ **ORB-SLAM3** (T-RO 2021) adds visual-inertial SLAM with an IMU (defined above), maximum a posteriori (MAP) estimation even during IMU initialization, and the multi-map **Atlas**: after tracking loss a new map starts, and it is merged once a mapped place is seen again.



Campos et al., "ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM," IEEE T-RO 2021, [arXiv:2007.11898v2](#), Figure 1.

Mur-Artal et al., "ORB-SLAM: a Versatile and Accurate Monocular SLAM System," IEEE T-RO 2015, [arXiv:1502.00956v2](#), Sec. III and Appendix, Eqs. 8–9.

Mur-Artal and Tardós, "ORB-SLAM2: an Open-Source SLAM System for Monocular, Stereo and RGB-D Cameras," IEEE T-RO 2017, [arXiv:1610.06475v2](#).

- **Idea:** skip descriptors; solve for the poses and depths that make high-gradient pixels agree in intensity across frames:

$$E_{\text{photo}} = \sum_i \sum_{\mathbf{p} \in \mathcal{P}_i} \sum_{j \in \text{obs}(\mathbf{p})} \left\| I_j(\pi(T_{ji} \pi^{-1}(\mathbf{p}, d_{\mathbf{p}}))) - I_i(\mathbf{p}) \right\|_{\gamma}$$

$\mathcal{P}_i$: pixels selected in frame i ; $\text{obs}(\mathbf{p})$: frames that see $\mathbf{p}$; $d_{\mathbf{p}} = 1/Z$: inverse depth; $\pi^{-1}(\mathbf{p}, d_{\mathbf{p}}) = d_{\mathbf{p}}^{-1} K^{-1} \tilde{\mathbf{p}}$: back-projection (defined above); $T_{ji} = T_j T_i^{-1}$: camera i to j ; $\|r\|_{\gamma}$: Huber norm (robust loss, defined above), threshold γ .

- **LSD-SLAM** (large-scale direct SLAM; Engel et al., ECCV 2014): keyframes hold semi-dense inverse depth maps (only near strong gradients), refined by filtering small-baseline stereo comparisons; frames are tracked by direct alignment; Sim(3) edges in a keyframe pose graph expose scale drift. Real time on a CPU.
- **DSO** (direct sparse odometry; Engel et al., TPAMI 2018): 2000 active points over all gradient regions, including edges and smooth intensity variations on white walls; poses, intrinsics and inverse depths optimized jointly in a sliding window; calibrated exposure time, vignetting (darkening toward the image corners) and camera response (light reaching the sensor to pixel value).

	Feature-based	Direct
Usable pixels	corners with descriptors	any pixel with gradient, even in low texture
Exposure changes	descriptors nearly invariant	modelled: calibration, per-frame affine gain and offset
Rolling shutter	tolerated (geometric error model)	accuracy drops quickly
Initialization	RANSAC on wide-baseline matches	needs a guess within 1–2 pixels

Rolling shutter: rows exposed in sequence; motion bends the image.

Engel et al., ECCV 2014, [doi:10.1007/978-3-319-10605-2_54](https://doi.org/10.1007/978-3-319-10605-2_54);

TPAMI 2018, [arXiv:1607.02565v2](https://arxiv.org/abs/1607.02565). Cadena et al., Sec. V.

- **Monocular:** depth, and with it scale, is not observable: trajectory and map are known up to a similarity, and the scale drifts.
- **Stereo and RGB-D:** the baseline B (defined above) or the depth sensor fixes metric scale.
- **Visual-inertial:** the IMU makes scale observable for a single camera; initialization also estimates the gravity direction and the IMU biases (ORB-SLAM3).
- **Absolute trajectory error (ATE):** align the estimated camera-to-world poses $\hat{Q}_i$ to the ground truth Q_i ($\hat{Q}_i, Q_i \in SE(3)$, $i = 1, \dots, n$) with the least-squares rigid transform A (closed form), then

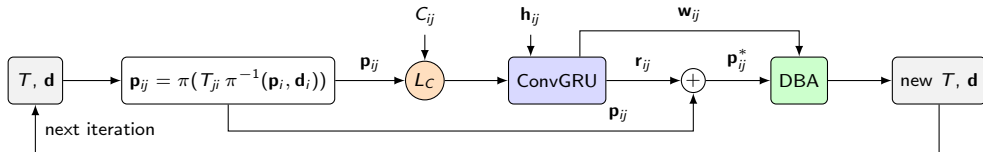
$$ATE = \left(\frac{1}{n} \sum_{i=1}^n \|\text{trans}(Q_i^{-1} A \hat{Q}_i)\|^2 \right)^{1/2},$$

the RMSE of the leftover translations. Monocular results use a 7-DoF similarity alignment, since scale is unknown.

- **Relative pose error (RPE):**
 $E_i = (Q_i^{-1} Q_{i+\Delta})^{-1} (\hat{Q}_i^{-1} \hat{Q}_{i+\Delta})$ compares the motion over a fixed interval Δ , so its RMSE measures drift ($\Delta = 30$ frames at 30 Hz gives drift per second).

Benchmark	Sensors and platform	Ground truth
TUM RGB-D (Sturm et al., 2012)	Kinect RGB-D, 640 × 480 at 30 Hz, hand-held or on a robot; 39 indoor sequences	motion capture, 8 cameras at 100 Hz
EuRoC MAV (Burri et al., 2016)	stereo and IMU on a micro aerial vehicle; 11 flights, from slow to fast with motion blur	laser tracker (mm position) or motion capture (6-DoF pose)
KITTI odometry (Geiger et al., 2012)	stereo cameras at 10 Hz on a car; 22 sequences, 39.2 km	GPS/IMU with real-time kinematic (RTK) correction

Sturm et al., "A Benchmark for the Evaluation of RGB-D SLAM Systems," IROS 2012, doi:[10.1109/IROS.2012.6385773](https://doi.org/10.1109/IROS.2012.6385773), Sec. VII.
 Burri et al., "The EuRoC Micro Aerial Vehicle Datasets," IJRR 2016, doi:[10.1177/0278364915620033](https://doi.org/10.1177/0278364915620033).
 Geiger et al., "Are We Ready for Autonomous Driving? The KITTI Vision Benchmark Suite," CVPR 2012, doi:[10.1109/CVPR.2012.6248074](https://doi.org/10.1109/CVPR.2012.6248074).
 Campos et al., [arXiv:2007.11898v2](https://arxiv.org/abs/2007.11898v2), Sec. II-B and V-B.



One iteration of the update operator for one edge (i, j) ; it runs on every edge of the frame graph. After Teed and Deng, “DROID-SLAM: Deep Visual SLAM for Monocular, Stereo, and RGB-D Cameras,” NeurIPS 2021, [arXiv:2108.10869v2](https://arxiv.org/abs/2108.10869v2), Figure 2.

- **State:** a pose $T_t \in SE(3)$ and a dense inverse-depth map $\mathbf{d}_t$ per frame. A frame graph links covisible frames; edges to old frames are added when the camera returns, which closes loops.
- **Update operator** (above): $\mathbf{p}_i$ is the pixel grid of frame i and $T_{ji} = T_j T_i^{-1}$. A lookup L_C in the correlation volume C_{ij} around $\mathbf{p}_{ij}$ and a 3×3 convolutional GRU with hidden state $\mathbf{h}_{ij}$ (both as in RAFT, defined above) predict a flow revision $\mathbf{r}_{ij}$ and a confidence $\mathbf{w}_{ij}$.
- **Beyond RAFT:** RAFT revises the flow between two frames; here the updates act on any number of frames, and a dense bundle adjustment (DBA) layer refines all poses and depth maps jointly, which the authors call essential for low drift on long trajectories and for loop closure.

- **DBA layer:** Gauss–Newton steps on all poses and depths, solved with the Schur complement (defined above), toward the revised targets $\mathbf{p}_{ij}^* = \mathbf{p}_{ij} + \mathbf{r}_{ij}$:

$$E(T', \mathbf{d}') = \sum_{(i,j) \in \mathcal{E}} \mathbf{e}_{ij}^\top \text{diag}(\mathbf{w}_{ij}) \mathbf{e}_{ij}$$

with $\mathbf{e}_{ij} = \mathbf{p}_{ij}^* - \pi(T'_{ji} \pi^{-1}(\mathbf{p}_i, \mathbf{d}'_i))$. The confidences act as the information matrix (Λ_{ij} above), so confident pixels count more.

- The layer is part of the computation graph, so training backpropagates through it end to end.

- **Data:** one model, trained only on monocular video from the synthetic TartanAir dataset (photo-realistic simulation), in 7-frame clips whose neighbouring frames move 8 to 96 pixels on average.
- **Loss:** a pose loss (distance between predicted and true poses) plus a flow loss (flow induced by the predicted against the true poses and depths), applied to all 15 unrolled iterations with weights growing toward the last ($\gamma = 0.9$). 250k steps, batch 4, 384×512 images: one week on four RTX 3090 GPUs.
- **Gauge:** in training the first two poses are fixed to ground truth: the first removes the 6-DoF gauge freedom, the second the scale.
- **Inference:** a front-end thread adds keyframes and runs local BA; a back-end thread runs global BA over all keyframes. Stereo uses both images with the left-right pose fixed; RGB-D adds a term on the measured depth. Real time needs two RTX 3090 GPUs.

TUM RGB-D fr1 (the 9 office sequences), RGB input only:

Method	Tracked	Mean ATE (m)
ORB-SLAM2	3 of 9	lost on 6
ORB-SLAM3	4 of 9	lost on 5
DeepFactors	9 of 9	0.233
DROID-SLAM	9 of 9	0.038

DeepFactors is a learned dense monocular SLAM system. Two baselines given extra information reach 0.116 m (RGB-D input) and 0.206 m (ground-truth scale).

- **EuRoC, monocular** (11 flights): 0.022 m mean ATE against 0.126 m for the best method without failures (82% lower); 43% lower than ORB-SLAM3 on the 10 flights it completes.
- **TartanAir test:** competition error 62% (mono) and 60% (stereo) lower than the best prior entry, while running 16 times faster.

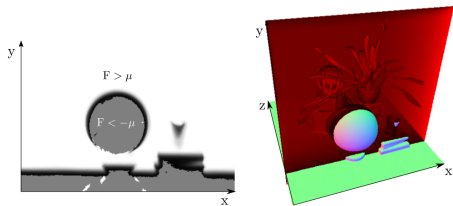
Teed and Deng, [arXiv:2108.10869v2](#), Sec. 3.3–4, Tables 2–4.
Czarnowski et al., “DeepFactors: Real-Time Probabilistic Dense Monocular SLAM,” IEEE RA-L 2020, [arXiv:2001.05049v1](#).
Wang et al., “TartanAir: A Dataset to Push the Limits of Visual SLAM,” IROS 2020, [arXiv:2003.14338v2](#).

- ▶ **Truncated signed distance function (TSDF)** (KinectFusion, Newcombe et al., ISMAR 2011): each voxel (3D grid cell) $\mathbf{p}$ stores the signed distance $F(\mathbf{p})$ to the nearest surface, positive in front and negative behind, truncated at $\pm\mu$, and a weight $W(\mathbf{p})$.
- ▶ **Fusion:** depth map D_k gives a TSDF F_{D_k} with weights W_{D_k} (distances measured along each pixel's ray), averaged into the volume:

$$F_k = \frac{W_{k-1}F_{k-1} + W_{D_k}F_{D_k}}{W_{k-1} + W_{D_k}}, \quad W_k = W_{k-1} + W_{D_k}$$

Setting $W_{D_k} = 1$ gives a plain average, which the authors found works well.

- ▶ **Surface and tracking:** the surface is the zero crossing of F . Raycasting (each pixel's ray marched to the zero crossing) predicts the surface seen from the last pose; the new depth frame is aligned to it by ICP (iterative closest point): each new point is paired with the predicted surface point it projects to, the pose that minimizes their distances along the surface normals (point-to-plane) is solved for, and the two steps repeat, coarse to fine, in real time on a GPU. Tracking against the fused model drifts less than frame-to-frame tracking.
- ▶ Cadena et al. (Sec. V) note that TSDFs became a popular map for vision-based SLAM after KinectFusion.

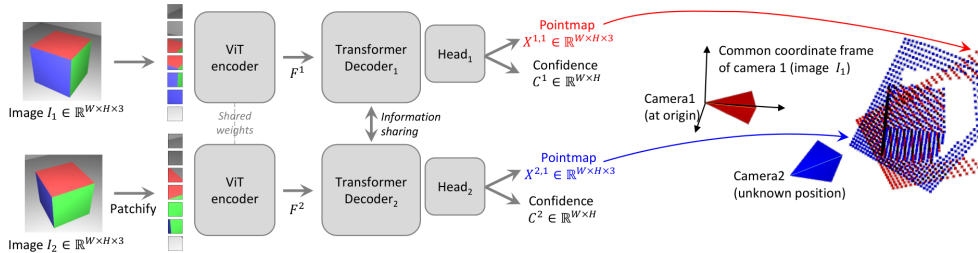


Left: a slice through the volume, with truncated free space $F > \mu$ (white), the smooth distance field around the surface $F = 0$, and voxels without a valid measurement yet (grey). Right: the scene, with the slice plane in red. Newcombe et al., "KinectFusion: Real-Time Dense Surface Mapping and Tracking," ISMAR 2011, [doi:10.1109/ISMAR.2011.6092378](https://doi.org/10.1109/ISMAR.2011.6092378), Figure 4.

- ▶ **NeRF** (neural radiance field; Mildenhall et al., ECCV 2020): an MLP maps a 3D position and viewing direction to density and colour; volume rendering sums the colours along each camera ray, weighted by density and by the transparency in front.
- ▶ **3D Gaussian splatting** (Kerbl et al., ACM TOG 2023): the scene is a set of anisotropic 3D Gaussians (stretched differently along each axis), each with a mean, covariance $\Sigma = R \text{diag}(\mathbf{s})^2 R^\top$ (R stored as a quaternion, defined above; $\mathbf{s}$ the 3D scale vector), opacity α , and view-dependent colour from spherical harmonics (a basis of functions on the sphere). A differentiable tile-based rasterizer projects each Gaussian to a 2D ellipse, sorts them by depth and alpha-blends them front to back, at 30 fps or more at 1080p.
- ▶ **Gaussian Splatting SLAM** (Matsuki et al., CVPR 2024): the first monocular SLAM system with Gaussians as its only map.
- ▶ **Tracking**: only the camera pose T_{CW} is optimized, to minimize $\|I(\mathcal{G}, T_{CW}) - \tilde{I}\|_1$, the L1 difference between the rendered Gaussians $\mathcal{G}$ and the frame $\tilde{I}$, using analytic Jacobians of the rendering with respect to the pose.
- ▶ **Mapping**: the Gaussians and the poses of a window of keyframes are optimized together; a regularizer pushes Gaussians toward isotropy, because rendering does not constrain their extent along the viewing ray.
- ▶ Runs live at 3 fps; with a depth sensor it adds a depth residual.



Center: rasterised Gaussians with camera poses, then shaded; sides: dotted-box close-ups (top rasterised, bottom shaded).
Matsuki et al., "Gaussian Splatting SLAM," CVPR 2024, [arXiv:2312.06741v2](https://arxiv.org/abs/2312.06741v2), Figure 1.



Wang et al., "DUST3R: Geometric 3D Vision Made Easy," CVPR 2024, [arXiv:2312.14132v3](https://arxiv.org/abs/2312.14132v3), Figure 2.

- **Pointmap:** an $H \times W \times 3$ array holding, for every pixel, the 3D point it sees, in one chosen camera's frame. With K and depth known it is back-projection, $X(\mathbf{p}) = D(\mathbf{p}) K^{-1} \tilde{\mathbf{p}}$.
- **Read-outs:** depth is the Z channel; the focal length is the f for which $f(X, Y)/Z$ best fits each pixel's offset from the image centre; relative pose comes from Procrustes (a closed-form similarity fit) between image 1's pointmaps from both input orders, or from PnP in RANSAC.
- **DUST3R:** two images in, with no intrinsics or poses; out: pointmaps $X^{1,1}, X^{2,1}$, both in camera 1's frame, and confidence maps $C^{1,1}, C^{2,1}$.

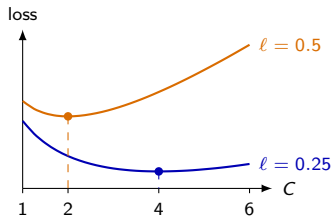
- **Encoder:** both images pass through one shared (Siamese) ViT-Large, giving token sets F^1 and F^2 .
- **Decoders:** two ViT-Base decoders, one per view. Every block applies self-attention within its view, then cross-attention to the other view, then an MLP, so the two branches exchange information throughout.
- **Heads:** a DPT head per view regresses the pointmap and its confidence map.
- **Confidence without labels:**
 $C = 1 + \exp \tilde{C} > 1$. With ℓ the regression error, the per-pixel term is smallest at $C = \alpha/\ell$ when $\ell < \alpha$, and keeps falling toward $C \rightarrow 1$ when $\ell \geq \alpha$, so confidence tracks inverse error (plot). Since $C > 1$, the network must still extrapolate in hard regions, such as areas seen by one view only.

Wang et al., CVPR 2024, [arXiv:2312.14132v3](https://arxiv.org/abs/2312.14132v3),
Sections 3.1–3.2 and 4, Eqs. 2–4.

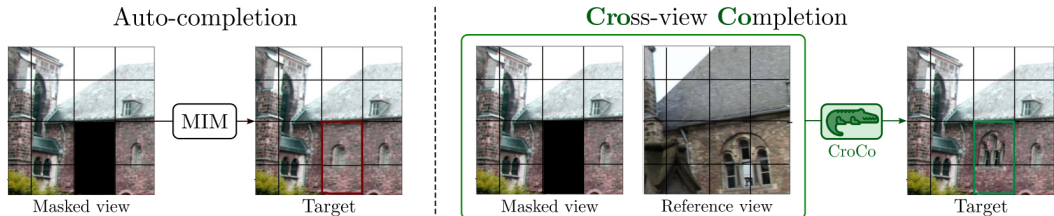
- **Confidence-aware regression loss:**

$$\mathcal{L}_{\text{conf}} = \sum_{v \in \{1,2\}} \sum_{i \in \mathcal{D}^v} C_i^{v,1} \left\| \frac{1}{z} X_i^{v,1} - \frac{1}{\bar{z}} \bar{X}_i^{v,1} \right\| - \alpha \log C_i^{v,1}$$

$X^{n,m}$: pointmap of image n in camera m 's frame; $\bar{X}$: ground truth; $\mathcal{D}^v$: pixels of view v with ground truth; $z, \bar{z}$: mean distance of the valid predicted and ground-truth points to the origin, which removes the unknown scale; α : regularizer weight.



Per-pixel term $C\ell - \alpha \log C$ for $\alpha = 1$, computed for this slide: both errors are below α , so the best C is α/ℓ , lower for the larger error.



Weinzaepfel et al., "CroCo: Self-Supervised Pre-training for 3D Vision Tasks by Cross-View Completion," NeurIPS 2022, [arXiv:2210.10716v2](https://arxiv.org/abs/2210.10716v2), Figure 1.

- ▶ **Left, auto-completion:** masked image modelling as in MAE (covered earlier). The visible patches alone leave the hidden content ambiguous, so the model leans on high-level semantics.
- ▶ **Right, cross-view completion:** the decoder also attends, through cross-attention, to a second, unmasked view of the same scene. With 90% of view 1's patches masked, that view resolves the ambiguity, provided the model relates the two viewpoints.
- ▶ **Use in DUST3R:** its network is inspired by this layout, with one decoder per view so that each view cross-attends to the other, and starts training from CroCo v2 weights (Weinzaepfel et al., ICCV 2023, [arXiv:2211.10408v3](https://arxiv.org/abs/2211.10408v3)).

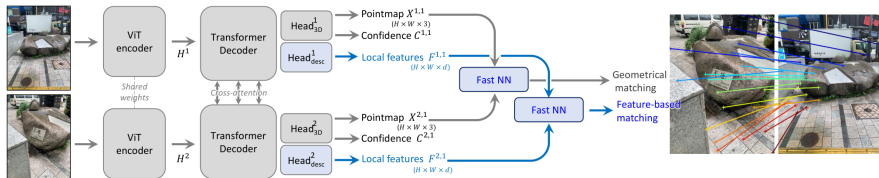
- ▶ **Pair graph:** images are vertices and overlapping pairs are edges $e = (n, m) \in \mathcal{E}$, found by image retrieval, or by running all pairs (about 40 ms each on an H100) and keeping the confident ones. The network predicts every edge.
- ▶ **Objective:** one world-frame pointmap χ^n per image, and one rigid transform $T_e \in SE(3)$ and one scale $\sigma_e > 0$ per pair, subject to $\prod_e \sigma_e = 1$, which rules out the trivial $\sigma_e = 0$:

$$\chi^* = \arg \min_{\chi, T, \sigma} \sum_{e \in \mathcal{E}} \sum_{v \in e} \sum_{i=1}^{HW} C_i^{v,e} \|\chi_i^v - \sigma_e T_e \chi_i^{v,e}\|$$

$\chi^{v,e}, C^{v,e}$: pair e 's pointmap and confidence for image v , both in the frame of image n , so one transform per pair aligns both of its pointmaps.

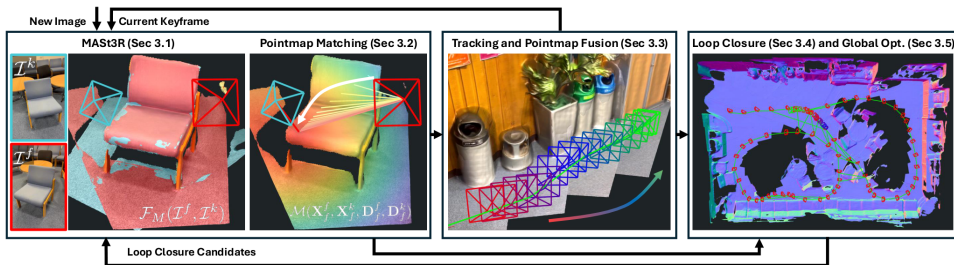
- ▶ **Cameras from the same problem:** writing $\chi^n(\mathbf{p}) = T_n^{-1}(D^n(\mathbf{p}) K_n^{-1} \tilde{\mathbf{p}})$ returns poses T_n , intrinsics K_n and depth maps D^n . The residual is a 3D distance, where bundle adjustment uses 2D reprojection error; gradient descent converges in a few hundred steps, seconds on one GPU.
- ▶ **Limits:** with all pairs the network runs $N(N-1)/2$ times (45 for 10 images), and the alignment is an offline optimization. Later work reports that it assumes a static scene and that DUS3R runs out of memory beyond 32 frames (CUT3R and VGGT, below).

Wang et al., CVPR 2024, [arXiv:2312.14132v3](#), Section 3.4, Eq. 5; static scene: [arXiv:2501.12387v1](#), Section 4.1; 32 frames: [arXiv:2503.11651v1](#), Figure 3.



New parts in blue. Leroy et al., “Grounding Image Matching in 3D with MASt3R,” ECCV 2024, [arXiv:2406.09756v1](https://arxiv.org/abs/2406.09756v1), Figure 2.

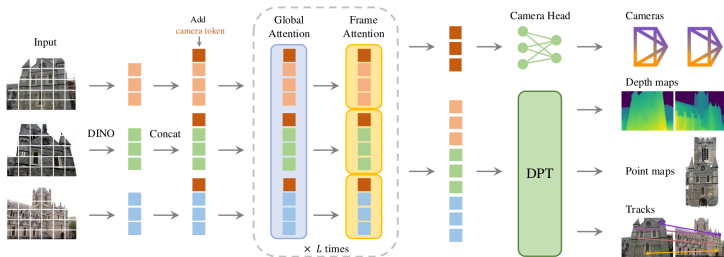
- **Descriptor head:** dense descriptors per image ($H \times W \times d$, $d = 24$), trained with an InfoNCE loss: each ground-truth match must pick out its partner among the other image’s candidate pixels, a classification in which a nearby pixel earns nothing.
- **Metric pointmaps:** no rescaling on metric ground truth (10 of 14 training sets): scale is learned.
- **Fast reciprocal matching:** from k grid pixels, hop to the nearest descriptor in the other image and back. Pixels that return to their start form reciprocal pairs; the rest repeat the hops from where they landed, and nearly all converge within 5 rounds. $O(kWH)$ work instead of $O(W^2H^2)$.
- **Map-free relocation** (one reference image, no map): VCRE AUC 0.933, against 0.697 for DUST3R and 0.634 for the best other published method (the authors’ “30% (absolute improvement)”). VCRE: virtual correspondence reprojection error, accepted below 90 pixels; AUC: area under the curve.



Murai et al., “MASt3R-SLAM: Real-Time Dense SLAM with 3D Reconstruction Priors,” CVPR 2025, [arXiv:2412.12392v2](https://arxiv.org/abs/2412.12392v2), Figure 3.

- **Idea:** real-time dense monocular SLAM built bottom-up on MAST3R’s two-view predictions; about 15 frames per second on one RTX 4090.
- **Camera model:** only a “generic central camera”, all rays through one centre. Each pointmap, normalized to unit rays, defines its own camera, so a changing zoom needs no calibration. Strong lens distortion still degrades MAST3R’s predictions; the EuRoC images were undistorted first.
- **Front-end:** matching by iterative projection onto these rays (2 ms, against 2 s for MAST3R’s dense matching), tracking against the current keyframe with a ray error, and pointmap fusion per keyframe. Poses live in Sim(3) because scale varies between predictions.
- **Back-end:** loop-closure candidates from retrieval on MAST3R encoder features, then Gauss–Newton optimization of all keyframe poses in Sim(3).

VGGT: One Transformer, Many 3D Outputs



Wang et al., “VGGT: Visual Geometry Grounded Transformer,” CVPR 2025 (Best Paper Award), [arXiv:2503.11651v1](https://arxiv.org/abs/2503.11651v1), Figure 2.

- **Input:** one, a few, or hundreds of images. DINOv2 (covered earlier) turns each frame into patch tokens, plus one camera token and four register tokens (extra learned tokens that collect global information and are discarded at the output); frame 1 has its own learned tokens, so every output is in camera 1’s frame.
- **Heads:** camera head: quaternion, translation and field of view (principal point centred); DPT head: depth maps and pointmaps with uncertainty, and features for point tracks (the pixel positions of one scene point across all frames).
- **Alternating attention:** $L = 24$ blocks, each with a frame-wise and a global self-attention layer; no cross-attention. About 1.2 billion parameters.
- **Speed:** about 0.2 s for 10 frames on one H100, against 7–9 s for DUST3R and MAST3R with global alignment. With optional BA the total is 1.8 s, and pose AUC@30 (accuracy integrated up to 30° of error) on RealEstate10K (about 80K YouTube clips with SLAM-derived poses) rises from 85.3 to 93.5.

Model	Venue	Input	Output	Key idea
CUT3R	CVPR 2025	video or photo collection, static or dynamic	metric pointmap and camera per frame, in one shared frame	a recurrent state is read and updated with each new frame: online, no global alignment
π^3	ICLR 2026	images in any order	per-view pose and local pointmap, up to one similarity	permutation-equivariant: no reference view, no frame-index embedding
MapAnything	3DV 2026	images, plus intrinsics, poses or depth if known	up-to-scale depth, ray directions and poses, plus one metric scale	factored output; one model for more than 12 tasks: SfM, localization, multi-view stereo (dense depth from posed images)
Depth Anything 3	ICLR 2026	any number of images, posed or not	depth and a per-pixel ray (origin and direction)	a plain DINOv2 transformer with cross-view attention; teacher–student training
VGGT- Ω	CVPR 2026	static and dynamic sequences	cameras and depth maps	register attention (frames interact only via register tokens) in 25% of global layers; up to 10B parameters

- **Reported gains**, on the authors' own evaluations: π^3 runs at 57.4 frames per second against 43.2 for VGGT (KITTI, one A800 GPU); Depth Anything 3 beats VGGT by 35.7% in pose and 23.6% in geometric accuracy on average; VGGT- Ω reaches a camera AUC@3 of 40.0 on Sintel (synthetic, with large moving objects), against 22.5 for the previous best.

CUT3R: Wang et al., [arXiv:2501.12387v1](#); π^3 : Wang et al., [arXiv:2507.13347v3](#); MapAnything: Keetha et al., [arXiv:2509.13414v3](#); Depth Anything 3: Lin et al., [arXiv:2511.10647v1](#); VGGT- Ω : Wang et al., [arXiv:2605.15195v1](#).

	Classical (SfM or SLAM, BA)	Feed-forward (DUST3R to VGGT- Ω)
Inputs	overlapping, textured images; intrinsics known or refined in BA	images only; MapAnything also accepts intrinsics, poses or depth, Depth Anything 3 known poses
Outputs	poses and sparse points; dense depth needs multi-view stereo	poses, intrinsics, dense depth or pointmaps; one forward pass from VGGT on
Speed, 10 images	COLMAP with SuperPoint and SuperGlue: about 15 s	VGGT 0.2 s; DUST3R or MAST3R with global alignment 7–9 s (all on one H100)
Metric scale	one camera: up to a similarity; scale from an IMU or known sizes	learned: MAST3R, CUT3R, MapAnything metric; DUST3R, VGGT, π^3 up to scale
Accuracy, AUC@10	on IMC (Image Matching Challenge): COLMAP with SIFT 44.79, up to 72.19 with learned matches; VGGsFmV2 (learned, with differentiable BA) 76.82	VGGT alone 71.26; streaming models trail optimization on MAST3R outputs

VGGT: [arXiv:2503.11651v1](#), Tables 1 and 10; MapAnything: [arXiv:2509.13414v3](#), Section 2; the other models are cited on the previous slides.

	Classical (SfM or SLAM, BA)	Feed-forward (DUST3R to VGGT- Ω)
Many images	sparse BA and pose graphs keep large maps consistent	VGGT backbone on one H100: 8.75 s and 40.6 GB for 200 frames; CUT3R may drift on very long sequences
Dynamic scenes	assume a static scene; large moving objects are hard	CUT3R, π^3 , VGGT- Ω train on dynamic data; VGGT fails on substantial non-rigid deformation
Unusual inputs	few views, little camera motion, non-Lambertian surfaces	outside the training data: fisheye, extreme rotation, motion blur, sudden zoom; transparent objects, since light transport is not modelled (π^3)

- **Combine them:** feed-forward output initializes, optimization refines. VGGT's cameras, points and point tracks start BA without triangulation, raising IMC AUC@10 from 71.26 to 84.91.
- MAST3R-SLAM wraps two-view MAST3R predictions in a keyframe SLAM back-end with loop closure and Sim(3) optimization, at about 15 frames per second.

VGGT [arXiv:2503.11651v1](#), Tables 9–10, Section 5; CUT3R [arXiv:2501.12387v1](#), Sections 4.2 and 5; VGGT- Ω [arXiv:2605.15195v1](#), Supplement C; π^3 [arXiv:2507.13347v3](#), Appendix A.8; DUST3R [arXiv:2312.14132v3](#), Section 1; MAST3R-SLAM [arXiv:2412.12392v2](#).

- ▶ **Surfaces that break the assumptions:** glass, mirrors and water break the Lambertian assumption (defined above), and large textureless walls lack the distinctive texture that matching, stereo, and photometric losses rely on.
- ▶ **Dynamic scenes:** most pipelines assume a static world. Moving people and vehicles corrupt poses and maps unless they are masked out or modelled explicitly.
- ▶ **Scale and unfamiliar cameras:** one camera recovers metric scale only through learned priors, an object of known size, or extra sensors, and a model that must infer the camera struggles with optics unlike its training data, such as fisheye or panoramic images.
- ▶ **Long-term operation:** lighting, season, and viewpoint changes degrade place recognition (at night, retrieve, match and solve localizes only 40.8% of Aachen queries within 0.5 m and 2°), and maps go stale as the world changes.
- ▶ **Cost of global attention:** multi-view transformers such as VGGT attend across the tokens of all views, so attention cost grows quadratically with the number of images; VGGT's backbone takes 3.12 s for 100 frames and 8.75 s for 200 on one H100 ([arXiv:2503.11651v1](#), Table 9). Recurrent models such as CUT3R avoid this but can drift on long sequences, so large scenes still need chunking, SfM, or a SLAM back-end.
- ▶ **Uncertainty and evaluation:** few learned models report calibrated uncertainty, and protocols that align predictions before scoring (median or scale-and-shift) hide the scale errors that matter to a robot.

- ▶ **One projection model underlies every method.** $\lambda \tilde{\mathbf{x}} = K[R \mid \mathbf{t}] \tilde{\mathbf{X}}$ has 5 intrinsic and 6 extrinsic DoF. Back-projection $\mathbf{X}_c = Z K^{-1} \tilde{\mathbf{x}}$ needs the depth Z , and without a baseline, a known size or an IMU the scene is fixed only up to scale. Distortion is removed first; Zhang's method calibrates K from the homographies of a planar pattern.
- ▶ **Two views constrain each other through E and F .** $\hat{\mathbf{x}}_2^\top E \hat{\mathbf{x}}_1 = 0$ with $E = [\mathbf{t}]_\times R$ (5 DoF) and $F = K_2^{-\top} E K_1^{-1}$ (7 DoF). Minimal solvers run inside RANSAC, whose sample count $\log(1-p)/\log(1-w^s)$ grows exponentially with s ; decomposing E gives the relative pose up to scale, and triangulation gives the points.
- ▶ **Stereo turns a match into depth:** $Z = fB/d$. Rectification reduces the search to one row, and the depth error per pixel of disparity error grows as $Z^2/(fB)$. A cost volume scores every candidate disparity; semi-global matching regularizes it along image paths, and learned stereo learns both the cost and its refinement.
- ▶ **One image does not fix depth: learned priors recover it up to scale, and the camera or familiar sizes supply the scale.** Scale-invariant losses (Eigen et al.), photometric self-supervision from video or stereo (Monodepth2), affine-invariant training on mixed data (MiDaS, DPT), camera-aware metric models (Metric3D, UniDepth, Depth Pro) and diffusion priors (Marigold) differ in their training signal; compare AbsRel and δ_1 only within one alignment protocol.

- ▶ **Absolute pose comes from 2D–3D matches: PnP inside RANSAC.** Visual localization retrieves, matches and solves. Pose regression behaves like image retrieval, while scene coordinate regression keeps the geometry in the solver. Incremental SfM interleaves registration, triangulation and bundle adjustment (Levenberg–Marquardt with the Schur complement); GLOMAP positions all cameras and points at once.
- ▶ **SLAM adds time and loop closure.** A front-end tracks features or pixels, a back-end optimizes keyframes, and loop closure with a Sim(3) pose graph removes drift. ORB-SLAM matches features, DSO aligns pixel intensities, DROID-SLAM learns its updates under a dense bundle adjustment layer, and Gaussian Splatting SLAM tracks against a rendered map.
- ▶ **Feed-forward models predict the geometry directly.** Pointmap networks (DUS_t3R, MAS_t3R) predict pointmaps for an image pair, from which cameras and depth are read out, and need global alignment for more views; multi-view transformers (VGGT, then π^3 , MapAnything, Depth Anything 3 and VGGT- Ω) output cameras and depth for all views in one pass. The most accurate results still come from refining them with bundle adjustment or a SLAM back-end.

1. Fischler and Bolles, "Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography," *Communications of the ACM* 24(6), 1981. doi:[10.1145/358669.358692](https://doi.org/10.1145/358669.358692)
2. Hartley, "In Defense of the Eight-Point Algorithm," *IEEE TPAMI* 19(6), 1997. doi:[10.1109/34.601246](https://doi.org/10.1109/34.601246)
3. Zhang, "A Flexible New Technique for Camera Calibration," *IEEE TPAMI* 22(11), 2000. doi:[10.1109/34.888718](https://doi.org/10.1109/34.888718); technical report MSR-TR-98-71, microsoft.com/en-us/research (source of Figure 4 and the Table 1 values)
4. Lowe, "Distinctive Image Features from Scale-Invariant Keypoints," *IJCV* 60(2), 2004. doi:[10.1023/B:VISI.0000029664.99615.94](https://doi.org/10.1023/B:VISI.0000029664.99615.94) (source of Figure 11)
5. Hirschmüller, "Stereo Processing by Semiglobal Matching and Mutual Information," *IEEE TPAMI* 30(2), 2008. doi:[10.1109/TPAMI.2007.1166](https://doi.org/10.1109/TPAMI.2007.1166)
6. Eigen, Puhrsch and Fergus, "Depth Map Prediction from a Single Image using a Multi-Scale Deep Network," NIPS 2014. arXiv:[1406.2283v1](https://arxiv.org/abs/1406.2283v1)
7. Mur-Artal, Montiel and Tardós, "ORB-SLAM: A Versatile and Accurate Monocular SLAM System," *IEEE Transactions on Robotics* 31(5), 2015. arXiv:[1502.00956v2](https://arxiv.org/abs/1502.00956v2)
8. Schönberger and Frahm, "Structure-from-Motion Revisited," CVPR 2016. doi:[10.1109/CVPR.2016.445](https://doi.org/10.1109/CVPR.2016.445) (source of Figure 2)
9. Godard et al., "Digging Into Self-Supervised Monocular Depth Estimation," ICCV 2019. arXiv:[1806.01260v4](https://arxiv.org/abs/1806.01260v4) (source of Figure 3(c))
10. Teed and Deng, "DROID-SLAM: Deep Visual SLAM for Monocular, Stereo, and RGB-D Cameras," NeurIPS 2021. arXiv:[2108.10869v2](https://arxiv.org/abs/2108.10869v2) (Figure 2 redrawn)
11. Wang et al., "DUST3R: Geometric 3D Vision Made Easy," CVPR 2024. arXiv:[2312.14132v3](https://arxiv.org/abs/2312.14132v3) (source of Figure 2)
12. Wang et al., "VGGT: Visual Geometry Grounded Transformer," CVPR 2025. arXiv:[2503.11651v1](https://arxiv.org/abs/2503.11651v1) (source of Figure 2 and Table 9)

13. Hartley and Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed., Cambridge University Press, 2004.
[doi:10.1017/CBO9780511811685](https://doi.org/10.1017/CBO9780511811685)
14. Brown, "Close-Range Camera Calibration," *Photogrammetric Engineering* 37(8), 1971. web.archive.org (ASPRS PDF)
15. OpenCV documentation, "Camera Calibration and 3D Reconstruction" (calib3d, 4.x).
docs.opencv.org/4.x/d9/d0c/group__calib3d.html
16. Rublee, Rabaud, Konolige and Bradski, "ORB: An Efficient Alternative to SIFT or SURF," ICCV 2011.
[doi:10.1109/ICCV.2011.6126544](https://doi.org/10.1109/ICCV.2011.6126544)
17. DeTone, Malisiewicz and Rabinovich, "SuperPoint: Self-Supervised Interest Point Detection and Description," CVPR Workshops 2018. [arXiv:1712.07629v4](https://arxiv.org/abs/1712.07629)
18. Sarlin et al., "SuperGlue: Learning Feature Matching with Graph Neural Networks," CVPR 2020. [arXiv:1911.11763v2](https://arxiv.org/abs/1911.11763)
19. Lindenberger, Sarlin and Pollefeys, "LightGlue: Local Feature Matching at Light Speed," ICCV 2023. [arXiv:2306.13643v1](https://arxiv.org/abs/2306.13643)
(source of Figure 1)
20. Sun et al., "LoFTR: Detector-Free Local Feature Matching with Transformers," CVPR 2021. [arXiv:2104.00680v1](https://arxiv.org/abs/2104.00680)
21. Edstedt et al., "RoMa: Robust Dense Feature Matching," CVPR 2024. [arXiv:2305.15404v2](https://arxiv.org/abs/2305.15404)
22. Nistér, "An Efficient Solution to the Five-Point Relative Pose Problem," *IEEE TPAMI* 26(6), 2004.
[doi:10.1109/TPAMI.2004.17](https://doi.org/10.1109/TPAMI.2004.17)
23. Fusiello, Trucco and Verri, "A Compact Algorithm for Rectification of Stereo Pairs," *Machine Vision and Applications* 12(1), 2000. [doi:10.1007/s001380050120](https://doi.org/10.1007/s001380050120)
24. Loop and Zhang, "Computing Rectifying Homographies for Stereo Vision," CVPR 1999. [doi:10.1109/CVPR.1999.786928](https://doi.org/10.1109/CVPR.1999.786928)
25. Geiger, Lenz, Stiller and Urtasun, "Vision Meets Robotics: The KITTI Dataset," *IJRR* 32(11), 2013.
[doi:10.1177/0278364913491297](https://doi.org/10.1177/0278364913491297); calibration files [2011.09.26_calib.zip](https://www.kitti.ai/2011/09/26/calib.zip)
26. Zabih and Woodfill, "Non-parametric Local Transforms for Computing Visual Correspondence," ECCV 1994.
[doi:10.1007/BFb0028345](https://doi.org/10.1007/BFb0028345)

27. Scharstein and Szeliski, "A Taxonomy and Evaluation of Dense Two-Frame Stereo Correspondence Algorithms," *IJCV* 47(1–3), 2002. [doi:10.1023/A:1014573219977](https://doi.org/10.1023/A:1014573219977)
28. Hirschmüller, "Semi-Global Matching: Motivation, Developments and Applications," Photogrammetric Week 2011, pp. 173–184. phowo.ifp.uni-stuttgart.de
29. OpenCV documentation, "cv::StereoSGBM Class Reference" (4.x). docs.opencv.org
30. Kendall et al., "End-to-End Learning of Geometry and Context for Deep Stereo Regression," ICCV 2017. [arXiv:1703.04309v1](https://arxiv.org/abs/1703.04309v1) (source of Figure 1)
31. Chang and Chen, "Pyramid Stereo Matching Network," CVPR 2018. [arXiv:1803.08669v1](https://arxiv.org/abs/1803.08669v1)
32. KITTI 2015 stereo benchmark (D1 metric). cvlibs.net/datasets/kitti
33. Teed and Deng, "RAFT: Recurrent All-Pairs Field Transforms for Optical Flow," ECCV 2020. [arXiv:2003.12039v3](https://arxiv.org/abs/2003.12039v3)
34. Lipson, Teed and Deng, "RAFT-Stereo: Multilevel Recurrent Field Transforms for Stereo Matching," 3DV 2021. [arXiv:2109.07547v1](https://arxiv.org/abs/2109.07547v1) (source of Figure 1)
35. Wen et al., "FoundationStereo: Zero-Shot Stereo Matching," CVPR 2025. [arXiv:2501.09898v4](https://arxiv.org/abs/2501.09898v4) (source of Figure 2)
36. Yang et al., "Depth Anything V2," NeurIPS 2024. [arXiv:2406.09414v2](https://arxiv.org/abs/2406.09414v2)
37. Sarbolandi, Lefloch and Kolb, "Kinect Range Sensing: Structured-Light versus Time-of-Flight Kinect," *CVIU* 139, 2015. [arXiv:1505.05459v1](https://arxiv.org/abs/1505.05459v1)
38. Li and Ibanez-Guzman, "Lidar for Autonomous Driving: The Principles, Challenges, and Trends for Automotive Lidar and Perception Systems," *IEEE Signal Processing Magazine* 37(4), 2020. [arXiv:2004.08467v1](https://arxiv.org/abs/2004.08467v1)
39. Silberman, Hoiem, Kohli and Fergus, "Indoor Segmentation and Support Inference from RGBD Images," ECCV 2012. [doi:10.1007/978-3-642-33715-4_54](https://doi.org/10.1007/978-3-642-33715-4_54)
40. Geiger, Lenz and Urtasun, "Are We Ready for Autonomous Driving? The KITTI Vision Benchmark Suite," CVPR 2012. [doi:10.1109/CVPR.2012.6248074](https://doi.org/10.1109/CVPR.2012.6248074)

41. Godard, Mac Aodha and Brostow, "Unsupervised Monocular Depth Estimation with Left-Right Consistency," CVPR 2017. [arXiv:1609.03677v3](#)
42. Zhou, Brown, Snavely and Lowe, "Unsupervised Learning of Depth and Ego-Motion from Video," CVPR 2017. [arXiv:1704.07813v2](#)
43. Wang, Bovik, Sheikh and Simoncelli, "Image Quality Assessment: From Error Visibility to Structural Similarity," *IEEE Transactions on Image Processing* 13(4), 2004. [doi:10.1109/TIP.2003.819861](#)
44. Ranftl et al., "Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-shot Cross-dataset Transfer," *IEEE TPAMI* 44(3), 2022. [arXiv:1907.01341v3](#)
45. Ranftl, Bochkovskiy and Koltun, "Vision Transformers for Dense Prediction," ICCV 2021. [arXiv:2103.13413v1](#) (source of Figure 1)
46. Yang et al., "Depth Anything: Unleashing the Power of Large-Scale Unlabeled Data," CVPR 2024. [arXiv:2401.10891v2](#)
47. Yin et al., "Metric3D: Towards Zero-shot Metric 3D Prediction from A Single Image," ICCV 2023. [arXiv:2307.10984v1](#)
48. Hu et al., "Metric3Dv2: A Versatile Monocular Geometric Foundation Model for Zero-shot Metric Depth and Surface Normal Estimation," *IEEE TPAMI* 2024. [arXiv:2404.15506v4](#)
49. Piccinelli et al., "UniDepth: Universal Monocular Metric Depth Estimation," CVPR 2024. [arXiv:2403.18913v1](#)
50. Bochkovskii et al., "Depth Pro: Sharp Monocular Metric Depth in Less Than a Second," ICLR 2025. [arXiv:2410.02073v2](#)
51. Ke et al., "Repurposing Diffusion-Based Image Generators for Monocular Depth Estimation," CVPR 2024. [arXiv:2312.02145v2](#) (source of Figure 2)
52. Gao, Hou, Tang and Cheng, "Complete Solution Classification for the Perspective-Three-Point Problem," *IEEE TPAMI* 25(8), 2003. [doi:10.1109/TPAMI.2003.1217599](#)
53. Lepetit, Moreno-Noguer and Fua, "EPnP: An Accurate $O(n)$ Solution to the PnP Problem," *IJCV* 81(2), 2009. [doi:10.1007/s11263-008-0152-6](#)

- 54. Sarlin, Cadena, Siegwart and Dymczyk, "From Coarse to Fine: Robust Hierarchical Localization at Large Scale," CVPR 2019. [arXiv:1812.03506v2](#) (source of Figure 2)
- 55. Arandjelović, Gronat, Torii, Pajdla and Sivic, "NetVLAD: CNN Architecture for Weakly Supervised Place Recognition," CVPR 2016. [arXiv:1511.07247v3](#)
- 56. Kendall, Grimes and Cipolla, "PoseNet: A Convolutional Network for Real-Time 6-DOF Camera Relocalization," ICCV 2015. [arXiv:1505.07427v4](#)
- 57. Sattler, Zhou, Pollefeys and Leal-Taixé, "Understanding the Limitations of CNN-based Absolute Camera Pose Regression," CVPR 2019. [arXiv:1903.07504v1](#)
- 58. Brachmann and Rother, "Visual Camera Re-Localization from RGB and RGB-D Images Using DSAC," *IEEE TPAMI* 2021. [arXiv:2002.12324v4](#)
- 59. Brachmann, Cavallari and Prisacariu, "Accelerated Coordinate Encoding: Learning to Relocalize in Minutes using RGB and Poses," CVPR 2023. [arXiv:2305.14059v1](#)
- 60. Huber, "Robust Estimation of a Location Parameter," *Annals of Mathematical Statistics* 35(1), 1964. [doi:10.1214/aoms/1177703732](#)
- 61. Levenberg, "A Method for the Solution of Certain Non-Linear Problems in Least Squares," *Quarterly of Applied Mathematics* 2(2), 1944. [doi:10.1090/qam/10666](#)
- 62. Marquardt, "An Algorithm for Least-Squares Estimation of Nonlinear Parameters," *Journal of the Society for Industrial and Applied Mathematics* 11(2), 1963. [doi:10.1137/0111030](#)
- 63. Triggs, McLauchlan, Hartley and Fitzgibbon, "Bundle Adjustment: A Modern Synthesis," *Vision Algorithms: Theory and Practice*, LNCS 1883, 2000. [doi:10.1007/3-540-44480-7_21](#)
- 64. Pan, Baráth, Pollefeys and Schönberger, "Global Structure-from-Motion Revisited," ECCV 2024. [arXiv:2407.20219v2](#) (source of Figure 2)

- 65. Cadena et al., "Past, Present, and Future of Simultaneous Localization And Mapping: Towards the Robust-Perception Age," *IEEE Transactions on Robotics* 32(6), 2016. [arXiv:1606.05830v4](#)
- 66. Strasdat, Montiel and Davison, "Scale Drift-Aware Large Scale Monocular SLAM," RSS 2010. [doi:10.15607/RSS.2010.VI.010](#)
- 67. Gálvez-López and Tardós, "Bags of Binary Words for Fast Place Recognition in Image Sequences," *IEEE Transactions on Robotics* 28(5), 2012. [doi:10.1109/TRO.2012.2197158](#)
- 68. Campos et al., "ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM," *IEEE Transactions on Robotics* 37(6), 2021. [arXiv:2007.11898v2](#) (source of Figure 1)
- 69. Mur-Artal and Tardós, "ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo and RGB-D Cameras," *IEEE Transactions on Robotics* 33(5), 2017. [arXiv:1610.06475v2](#)
- 70. Engel, Schöps and Cremers, "LSD-SLAM: Large-Scale Direct Monocular SLAM," ECCV 2014. [doi:10.1007/978-3-319-10605-2_54](#)
- 71. Engel, Koltun and Cremers, "Direct Sparse Odometry," *IEEE TPAMI* 40(3), 2018. [arXiv:1607.02565v2](#)
- 72. Sturm et al., "A Benchmark for the Evaluation of RGB-D SLAM Systems," IROS 2012. [doi:10.1109/IROS.2012.6385773](#)
- 73. Burri et al., "The EuRoC Micro Aerial Vehicle Datasets," *IJRR* 35(10), 2016. [doi:10.1177/0278364915620033](#)
- 74. Wang et al., "TartanAir: A Dataset to Push the Limits of Visual SLAM," IROS 2020. [arXiv:2003.14338v2](#)
- 75. Czarowski et al., "DeepFactors: Real-Time Probabilistic Dense Monocular SLAM," *IEEE Robotics and Automation Letters* 5(2), 2020. [arXiv:2001.05049v1](#)
- 76. Newcombe et al., "KinectFusion: Real-Time Dense Surface Mapping and Tracking," ISMAR 2011. [doi:10.1109/ISMAR.2011.6092378](#) (source of Figure 4)
- 77. Mildenhall et al., "NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis," ECCV 2020. [arXiv:2003.08934v2](#)

- 78. Kerbl, Kopanas, Leimkühler and Drettakis, “3D Gaussian Splatting for Real-Time Radiance Field Rendering,” *ACM Transactions on Graphics* 42(4), 2023. [arXiv:2308.04079v1](#)
- 79. Matsuki, Murai, Kelly and Davison, “Gaussian Splatting SLAM,” CVPR 2024. [arXiv:2312.06741v2](#) (source of Figure 1)
- 80. Weinzaepfel et al., “CroCo: Self-Supervised Pre-training for 3D Vision Tasks by Cross-View Completion,” NeurIPS 2022. [arXiv:2210.10716v2](#) (source of Figure 1)
- 81. Weinzaepfel et al., “CroCo v2: Improved Cross-view Completion Pre-training for Stereo Matching and Optical Flow,” ICCV 2023. [arXiv:2211.10408v3](#)
- 82. Wang et al., “Continuous 3D Perception Model with Persistent State,” CVPR 2025. [arXiv:2501.12387v1](#)
- 83. Leroy, Cabon and Revaud, “Grounding Image Matching in 3D with MAST3R,” ECCV 2024. [arXiv:2406.09756v1](#) (source of Figure 2)
- 84. Murai, Dexheimer and Davison, “MASt3R-SLAM: Real-Time Dense SLAM with 3D Reconstruction Priors,” CVPR 2025. [arXiv:2412.12392v2](#) (source of Figure 3)
- 85. Wang et al., “ π^3 : Permutation-Equivariant Visual Geometry Learning,” ICLR 2026. [arXiv:2507.13347v3](#)
- 86. Keetha et al., “MapAnything: Universal Feed-Forward Metric 3D Reconstruction,” 3DV 2026. [arXiv:2509.13414v3](#)
- 87. Lin et al., “Depth Anything 3: Recovering the Visual Space from Any Views,” ICLR 2026. [arXiv:2511.10647v1](#)
- 88. Wang et al., “VGGT- Ω ,” CVPR 2026. [arXiv:2605.15195v1](#)
- 89. Wang et al., “Visual Geometry Grounded Deep Structure From Motion,” CVPR 2024. [arXiv:2312.04563v1](#)